

Database Development with PL/SQL INSY 8311



ADVENTIST UNIVERSITY
OF CENTRAL AFRICA

Instructor:

- Eric Maniraguha | ericmaniraguha2024@gmail.com | [LinkedIn Profile](#)



6h00 pm – 9h50 pm

- Monday A -G207
- Tuesday B-G204
- Wednesday E-G207
- Thursday F-G307

January 2025

Database Development with PL/SQL



Reference reading

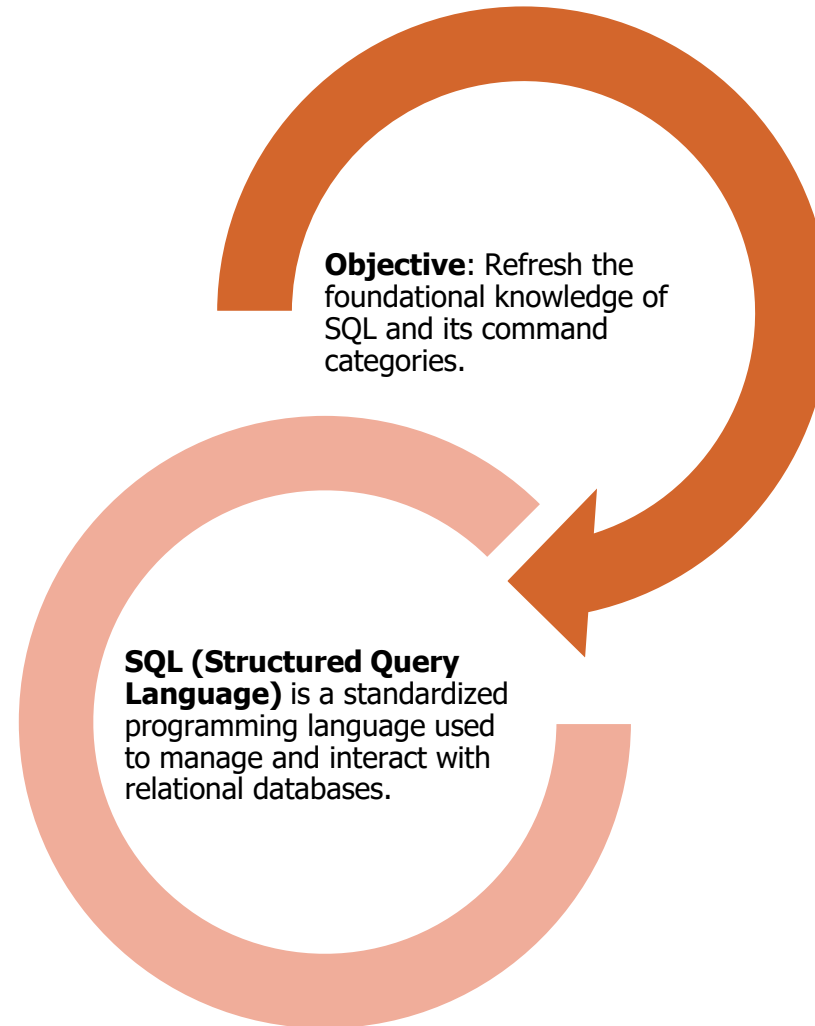
- [What is a relational database?](#)
- [What is RDBMS\(Relational Database Management System\)?](#)
- [MySQL RDBMS](#)
- [Normalization in SQL DBMS: 1NF, 2NF, 3NF, and BCNF Examples](#)
- [Hacker Rank](#) : Skills speak louder than words
- [SQL indexing best practices | How to make your database FASTER!](#)
- [Normal Forms Introduction](#)
- [SQL Window Functions](#)

Lecture 01 - Introduction to SQL Command Basics (Recap)

Oracle SQL Basics



What is SQL?



Types of SQL Commands (Recap)



Objective: Refresh the foundational knowledge of SQL and its command categories.

SQL is a powerful tool primarily used for **querying** and **manipulating data** within databases. It enables users to:

- **Insert** new data.
- **Update** existing records.
- **Delete** unnecessary or outdated information.
- **Retrieve** data efficiently.

In addition to handling data, SQL is also used to **define and modify database structures**—including tables, indexes, and constraints—ensuring smooth database management.

Data Definition Language (DDL):

1. Used to define or modify the **structure** of a database.
2. Includes commands:
 - **CREATE** – Creates new database objects (e.g., tables).
 - **ALTER** – Modifies existing database structures.
 - **DROP** – Deletes database objects.
 - **TRUNCATE** – Removes all records from a table but keeps its structure.

Data Manipulation Language (DML):

1. Focuses on **manipulating data** stored in database objects.
2. Includes commands:
 - **INSERT** – Adds new records to a table.
 - **UPDATE** – Modifies existing records.
 - **DELETE** – Removes records from a table.

Data Query Language (DQL):

1. Used to **query and retrieve data** from the database.
2. Includes the command:
 - **SELECT** – Fetches data based on specified criteria.

Data Control Language (DCL):

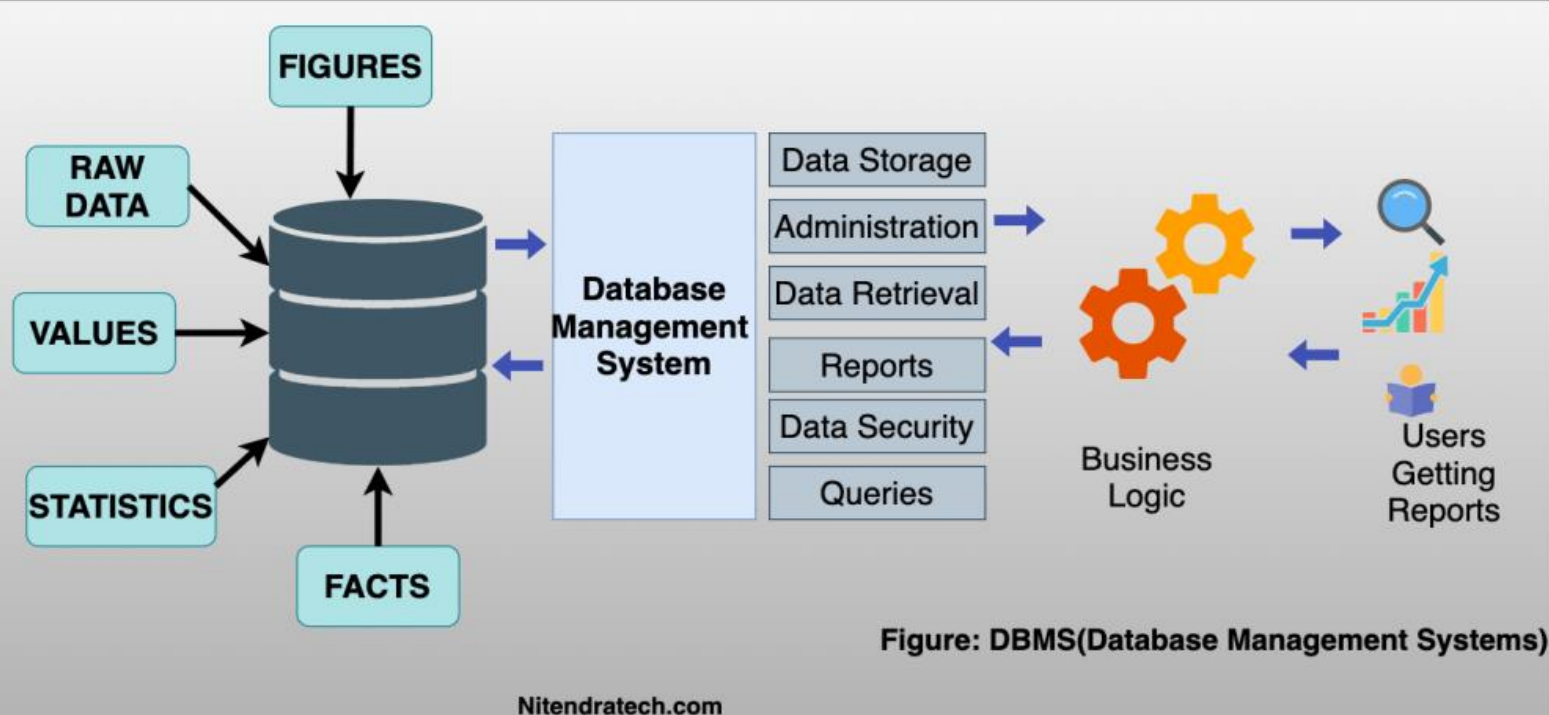
1. Manages **permissions and access control** within the database.
2. Includes commands:
 - **GRANT** – Assigns permissions to users.
 - **REVOKE** – Removes permissions from users.

Understanding RDBMS

What is RDBMS?

A **Relational Database Management System (RDBMS)** is a database management system that uses a relational model to organize and store data. In a relational database, data is organized into tables, also known as relations, which consist of rows and columns. Each row represents a single record, and each column represents a specific data field. RDBMS uses a structured query language (SQL) to access and manipulate the data stored in the tables. SQL allows users to insert, update, delete and query data in the tables. It also allows creating, altering, and deleting tables and other database objects.

A Relational database management system (RDBMS) is a database management system (DBMS) that is based on the relational model as introduced by E. F. Codd in 1970.



Database Management System (DBMS): Workflow, Components, and Functionality

The previous diagram illustrates the architecture and workflow of a **Database Management System (DBMS)**. Here's a detailed explanation of each component:

1. Input Data Sources:

RAW DATA: Unprocessed data collected from various sources.

VALUES: Specific numerical or textual values relevant to the database.

STATISTICS: Summarized or analytical data used for decision-making.

FACTS: Verified information used for further processing.

FIGURES: Numerical representations that can be analyzed or visualized.

These inputs are stored in a **Database Management System (DBMS)**.

2. Database Management System (DBMS): The core of the system that manages and processes the data. It includes the following functionalities:

Data Storage: Organizes and stores raw data, statistics, and facts in structured formats.

Administration: Handles database configuration, user permissions, and performance monitoring.

Data Retrieval: Allows querying to extract specific data based on requirements.

Reports: Generates summaries and structured outputs for decision-making.

Data Security: Ensures data privacy, integrity, and access control.

Queries: Processes user queries to fetch relevant information efficiently.

3. Business Logic:

Applies rules and algorithms to process data retrieved from the DBMS.

Transforms data into meaningful insights or actionable outputs.

4. Output for Users:

Users Getting Reports: End-users receive analytical reports and visual insights, such as graphs and charts, to support decision-making.

Summary:

The DBMS collects, organizes, and processes data from multiple inputs (raw data, values, statistics, etc.) to generate meaningful reports. These outputs are used by business logic systems and end-users to analyze trends, create insights, and make data-driven decisions.

Understanding RDBMS - Table

What is a Table?

The data in an RDBMS is stored in database objects known as **tables**. This table is basically a collection of related data entries and it consists of numerous columns and rows.



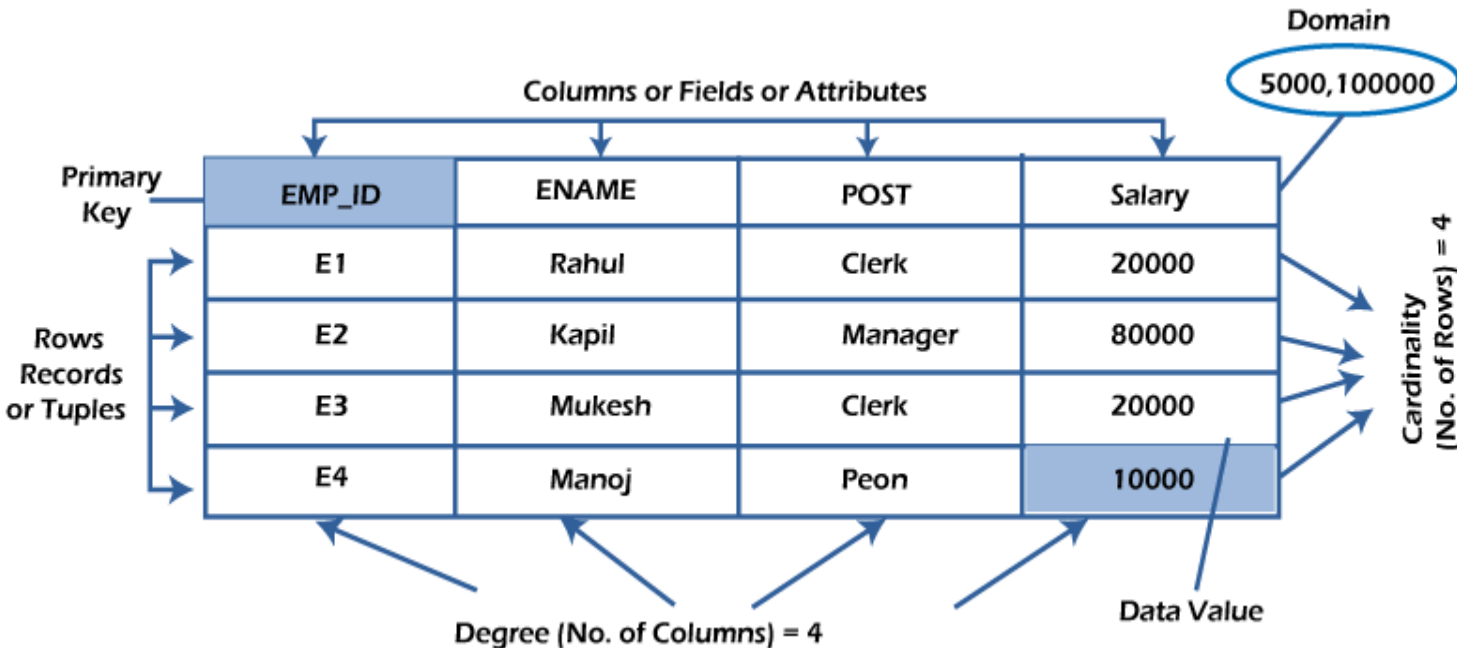
Remember, a table is the most common and simplest form of data storage in a relational database. Following is an example of a CUSTOMERS table which stores customer's ID, FirstName, LastName, Birthdate.

The diagram shows a table with four columns: CustomerID, FirstName, LastName, and Birthdate. It contains four rows of data. Annotations with red arrows and boxes identify key components: 'Column (attribute)' points to the 'FirstName' header; 'Table (relation)' points to the entire table structure; 'Row (tuple)' points to the first data row; 'Primary key' points to the 'CustomerID' column; and 'Data value' points to the 'Green' value in the 'LastName' column of the fourth row.

CustomerID	FirstName	LastName	Birthdate
XY001	John	Doe	April 18, 1929
BR092	Mary	Green	March 4, 1980
PD500	Francesca	de la Gillebert	September 12, 1959
WI308	John	Green	March 4, 1980

Source Image: <https://www.c-sharpcorner.com/article/sql-server-and-relational-database-part-one/>

RDBMS Terminologies



Source Image: <https://www.javatpoint.com/what-is-rdbms>

Properties of a row:

- No two tuples are identical to each other in all their entries.
- All tuples of the relation have the same format and the same number of entries.
- The order of the tuple is irrelevant. They are identified by their content, not by their position.

Properties of an Attribute:

- Every attribute of a relation must have a name.
- Null values are permitted for the attributes.
- Default values can be specified for an attribute automatically inserted if no other value is specified for an attribute.
- Attributes that uniquely identify each tuple of a relation are the primary key.

Difference Between RDBMS and NoSQL (Non-Relational Databases)

Definition

- **Relational Database (RDBMS):**

Data is stored in **tables** with rows and columns, where relationships between data are defined using **keys**.

- **Non-relational Database (NoSQL):**

Data is stored in **hierarchical, document-based, key-value pairs, or graph structures**, making it flexible for **unstructured, semi-structured, and structured data**.



Difference between Relational and Non-Relational databases

Relational

student

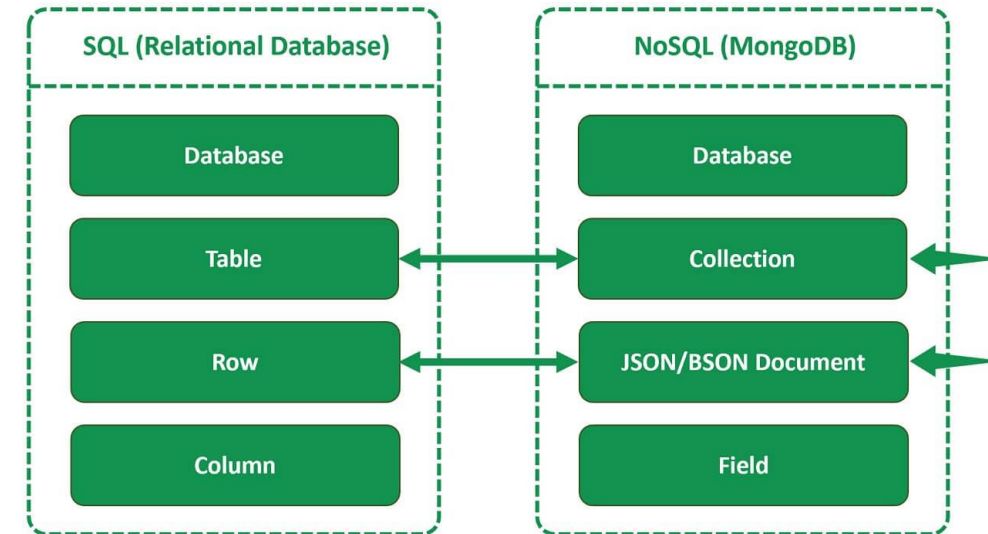
id	name	surname	age
1	John	Brown	19
2	Emma	Carly	23

Non-Relational

student.json file body:

```
[
  {
    "id": 1,
    "name": "John",
    "surname": "Brown",
    "age": 19
  },
  {
    "id": 2,
    "name": "Emma",
    "surname": "Carly",
    "age": 23
  }
]
```

Source Image: Data in non-relational databases often resemble **dictionary-like structures**, akin to those in Python. Below is an example showcasing this relationship:



Examples of NoSQL Formats: BSON and JSON

BSON is a binary encoded Javascript Object Notation (JSON)

```
{ "hello": "world" } →  
\x16\x00\x00\x00      // total document size  
\x02                   // 0x02 = type String  
hello\x00              // field name  
\x06\x00\x00\x00world\x00 // field value  
\x00                   // 0x00 = type E00 ('end of object')  
  
{"BSON": ["awesome", 5.05, 1986]} →  
\x31\x00\x00\x00  
\x04BSON\x00  
\x26\x00\x00\x00  
\x02\x30\x00\x08\x00\x00\x00awesome\x00  
\x01\x31\x00\x33\x33\x33\x33\x33\x33\x14\x40  
\x10\x32\x00\xc2\x07\x00\x00  
\x00  
\x00
```

JSON (JavaScript Object Notation)

```
{  
  "_id": 1,  
  "name": { "first" : "John", "last" : "Backus" },  
  "contribs": [ "Fortran", "ALGOL", "Backus-Naur Form", "FP" ],  
  "awards": [  
    {  
      "award": "W.W. McDowell Award",  
      "year": 1967,  
      "by": "IEEE Computer Society"  
    }, {  
      "award": "Draper Prize",  
      "year": 1993,  
      "by": "National Academy of Engineering"  
    }  
  ]  
}
```



NoSQL databases offer greater schema flexibility than RDBMS, which can be advantageous in certain use cases. For example, suppose you are implementing an IoT platform that stores data from different kinds of sensors.



Relational vs Non-Relational Databases: Key Features and Differences

Criteria	Relational Database (RDBMS)	Non-relational Database (NoSQL)
Stored Items	Items are related through keys and constraints.	Items can be structured, unstructured, or semi-structured .
Data Integrity	High due to relationships and constraints like primary and foreign keys.	Lower integrity since relationships are not enforced by constraints.
Scalability	Vertical scaling (adding resources to a single server) is expensive.	Horizontal scaling (adding more servers) is cheaper and faster.
Storage Capacity	Suitable for medium to large datasets.	Ideal for big data and high-volume data storage.
Reliability	Highly reliable due to ACID (Atomicity, Consistency, Isolation, Durability) compliance.	Less reliable as it may not support ACID properties.
Flexibility	Requires fixed schemas for data storage and retrieval.	Schema-less, allowing dynamic changes in data models.
Performance	Slower for complex queries involving joins.	Faster performance for queries on large datasets.
Query Language	Uses SQL (Structured Query Language) for data manipulation.	Uses NoSQL-specific languages or APIs.
Data Processing	Best for transactional systems requiring structured relationships.	Best for real-time analytics and handling diverse data types.
Backup and Consistency	Easier to maintain backups and consistency .	More complex backup and consistency management.

RDBMS vs NoSQL: Pros, Cons, and Key Use Cases

1. Advantages (Pros)

RDBMS



- ☐ **Data Integrity** – Enforces constraints for accuracy.
- ☐ **Data Accuracy** – Ensures relationships and consistency.
- ☐ **Normalization** – Minimizes redundancy.
- ☐ **Simple Structure** – Easy to understand with tabular format.
- ☐ **Secure** – Robust security and user access controls.
- ☐ **Multi-user Support** – Allows concurrent access by multiple users.

NoSQL



- ☐ **Handles Unstructured Data** – Suitable for JSON, XML, and multimedia files.
- ☐ **High Performance** – Optimized for high-speed queries.
- ☐ **Scalable** – Handles big data with horizontal scaling.
- ☐ **Flexible Schema** – Supports schema-less models, ideal for dynamic changes.
- ☐ **Open-Source** – Cost-effective and widely supported.

2. Disadvantages (Cons)

RDBMS

Slow Processing – Poor performance with **complex data types**.

Rigid Schema – Requires predefined structure, limiting flexibility.

Expensive Scalability – Vertical scaling demands costly hardware.

NoSQL

Limited Functionality – Does not support **complex transactions** well.

Manual Query Language – May require programming expertise for queries.

Data Consistency – Weak consistency models compared to RDBMS.

Backup Issues – Managing backups and maintaining consistency can be challenging.

3. Use Cases

RDBMS Examples:

MySQL, SQL Server, Oracle Database, PostgreSQL – Best for transactional systems like **banking, inventory, and CRM systems**.

NoSQL Examples:

MongoDB, Cassandra, CouchDB, DocumentDB – Ideal for applications handling **big data, IoT, content management systems, and real-time analytics**.

Key Features of RDBMS (Relational Database Management System)



Structured Storage

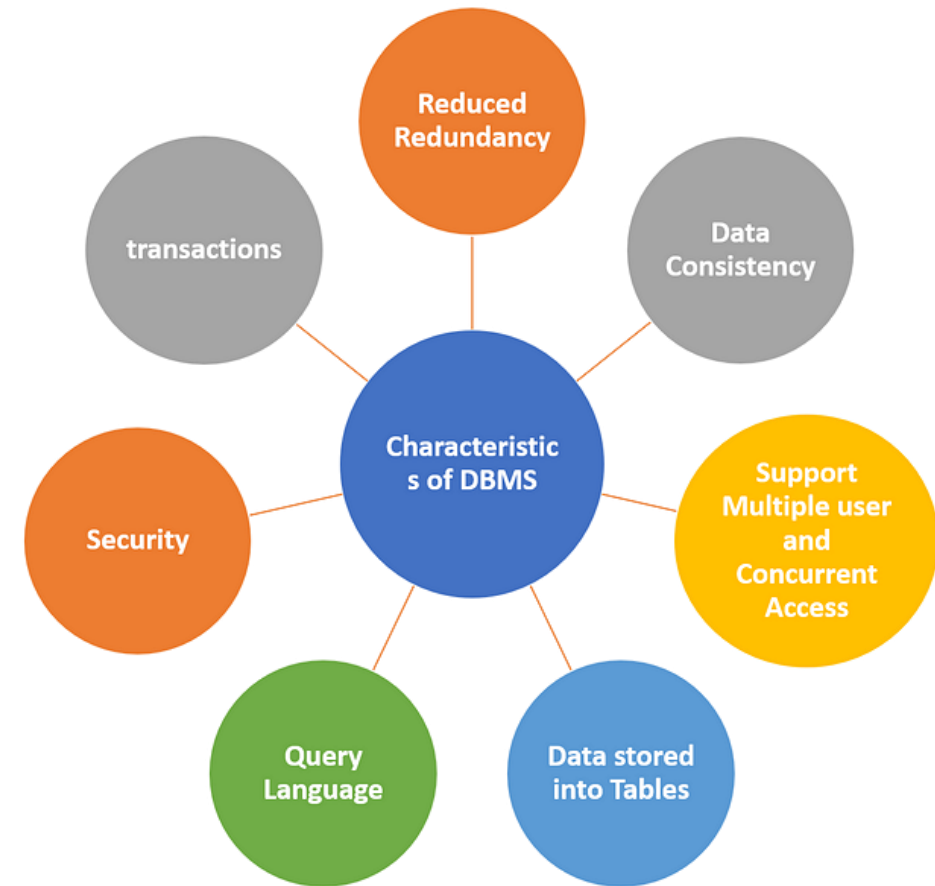
1. Organizes data into **tables** using rows and columns.
2. Supports **relationships** between tables through **keys** for organized access.

Indexing

1. Improves **data retrieval speed** by creating optimized **search indexes**.
2. Enhances performance for queries and lookups.

Virtual Tables (Views)

1. Allows creation of **virtual tables** to simplify query execution.
2. Protects **sensitive data** by providing restricted views of the database.



Source Image: <https://medium.com/@tharshamohan2000/why-rdbms-why-dbms-6405e66ad2cf>

ACID Properties in DBMS - Student Notes

1. Atomicity

- Ensures transactions are all-or-nothing.
- **Abort:** Rolls back changes if the transaction fails.
- **Commit:** Saves changes if the transaction succeeds.

Example: Transferring 100 from X to Y—either both debit and credit happen, or none.

2. Consistency

- Maintains database correctness before and after a transaction.
- Integrity constraints must always be satisfied.

Example: Total balance before and after transfer must remain unchanged.

3. Isolation

- Ensures transactions execute independently.
- Changes are not visible to others until committed.

Example: Concurrent transactions must avoid interference to prevent inconsistent results.

4. Durability

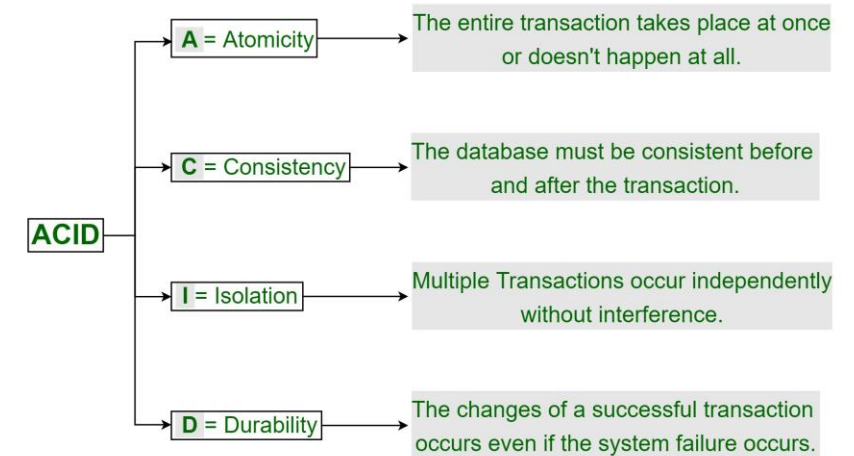
- Guarantees committed transactions are permanent.
- Data is saved even during system failures.

Example: Bank transfers remain recorded even if the system crashes.

Summary ACID properties—Atomicity, Consistency, Isolation, and Durability—are essential for reliable and consistent database transactions.



ACID Properties in DBMS



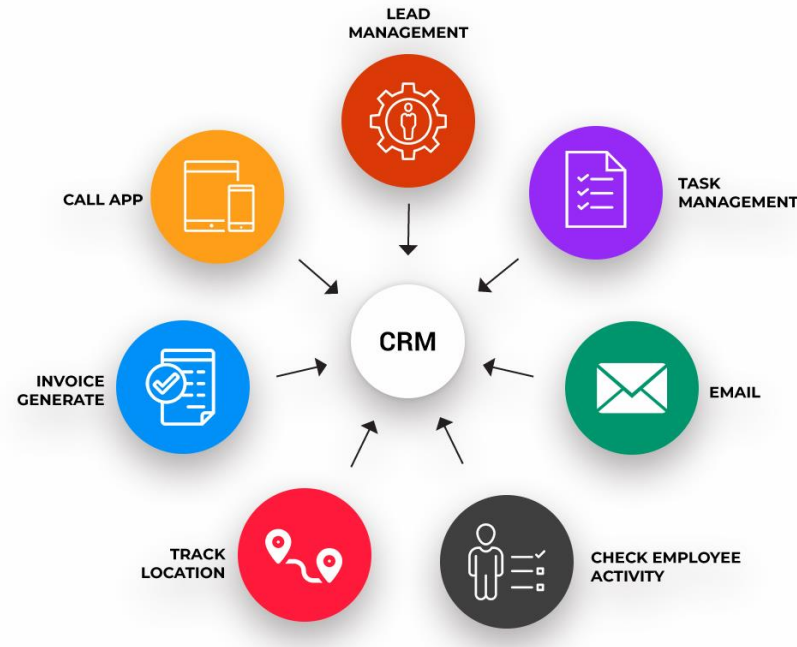
Source Image: <https://www.geeksforgeeks.org/acid-properties-in-dbms>



Real-World Applications of the Relational Model



Customer Relationship Management System



Key Differences Between Primary Key , Foreign Key and Indexes			
Aspect	Primary Key	Foreign Key	Index
Purpose	Enforces data integrity by ensuring uniqueness.	Establishes a relationship between tables.	Speeds up query performance, improves data retrieval.
Uniqueness	Yes, values must be unique.	No, values can repeat.	Optional, values can be unique or non-unique.
Null Values	No, must not contain NULL values.	Yes, unless the foreign key is part of a unique constraint.	Yes, NULL values are allowed unless part of unique index.
Index Creation	Automatically created when a Primary Key is defined.	No automatic index creation, index must be created explicitly.	Index must be explicitly created for query optimization.
Enforcement of Data Integrity	Yes, ensures that no duplicate or NULL values exist in the column.	Ensures referential integrity between tables.	No, it is used for optimization purposes only.
Relation to Tables	Unique identifier for rows in the same table.	Links a column in one table to the Primary Key of another table.	Does not enforce any relationship between tables.
Number Allowed per Table	Only one Primary Key per table.	Multiple Foreign Keys can exist in a table.	Multiple indexes can be created per table.
Automatic Behavior	Automatically enforces uniqueness and integrity.	Enforces referential integrity only if explicitly defined.	Must be manually defined by the user.

SQL Operations: Primary Key, Foreign Key, and Index

Operation	Primary Key (SQL Query)	Foreign Key (SQL Query)	Index (SQL Query)
Create	CREATE TABLE Customers (CustomerID INT PRIMARY KEY, Name VARCHAR(100));	CREATE TABLE Orders (OrderID INT PRIMARY KEY, CustomerID INT, FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID));	CREATE INDEX idx_lastname ON Customers (LastName);
Delete	DELETE FROM Customers WHERE CustomerID = 101;	DELETE FROM Orders WHERE OrderID = 150;	DROP INDEX idx_lastname ON Customers;
Delete Primary Key	ALTER TABLE Customers DROP CONSTRAINT PK_Customers;		
Delete Foreign Key		ALTER TABLE Orders DROP CONSTRAINT FK_Orders_Customers;	
Add Primary Key	ALTER TABLE Customers ADD CONSTRAINT PK_Customers PRIMARY KEY (CustomerID);		
Add Foreign Key		ALTER TABLE Orders ADD CONSTRAINT FK_Orders_Customers FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID);	
Update	UPDATE Customers SET CustomerID = 102 WHERE CustomerID = 101;	UPDATE Orders SET CustomerID = 200 WHERE OrderID = 150;	(No direct update; indexes are automatically updated as data changes)

Data normalization

Data normalization is the process of organizing a database to reduce redundancy and improve data integrity. **It involves structuring the database in a way that minimizes duplication of data and ensures that the data is stored efficiently. The goal is to ensure that the data is stored in such a way that it is easy to maintain, update, and query.** The process of normalization typically follows a series of steps called **normal forms**. Each normal form builds upon the previous one to ensure that the database structure is optimized.

Boyce Codd Normal Form (BCNF) or 3.5 NF

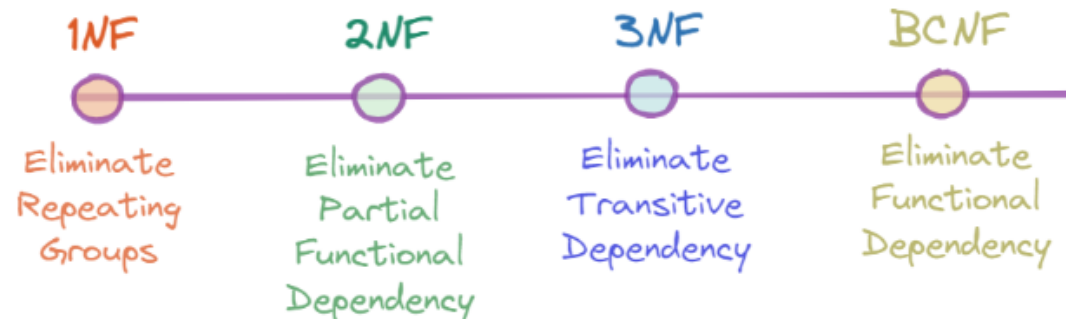
The **Boyce Codd Normal Form** is also known as **3.5NF** since it is a higher version of 3NF which was developed to handle specific sorts of anomalies that 3NF did not solve. Once again, the table *must* fulfill the 3rd Normal Form before continuing to BCNF. Moreover, every *Right-Hand Side (RHS)* attribute of the functional dependencies should be dependent on the table's super key. Now, let's look at an *example* in order to better understand the principle of BCNF:

Key Concepts in Data Normalization:

Eliminate Redundant Data: Store data in such a way that the same information is not repeated in multiple places. This reduces the risk of inconsistency.

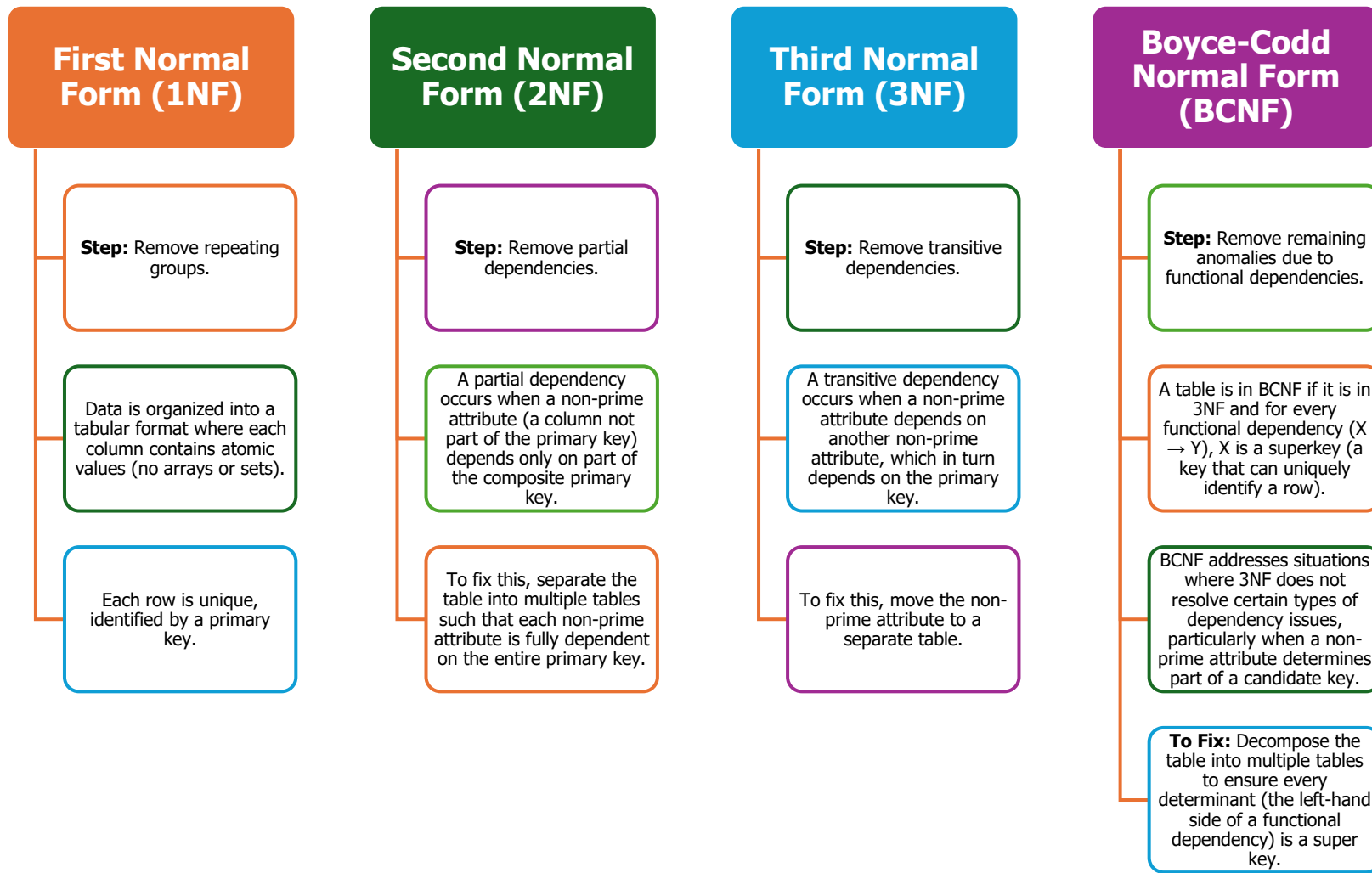
Ensure Data Integrity: By organizing data into logical units (tables), normalization ensures that changes to data are consistent across the system and **prevents anomalies like insert, update, or delete anomalies.**

Improve Query Efficiency: A well-normalized database allows for more efficient queries by reducing the amount of unnecessary data that needs to be processed.



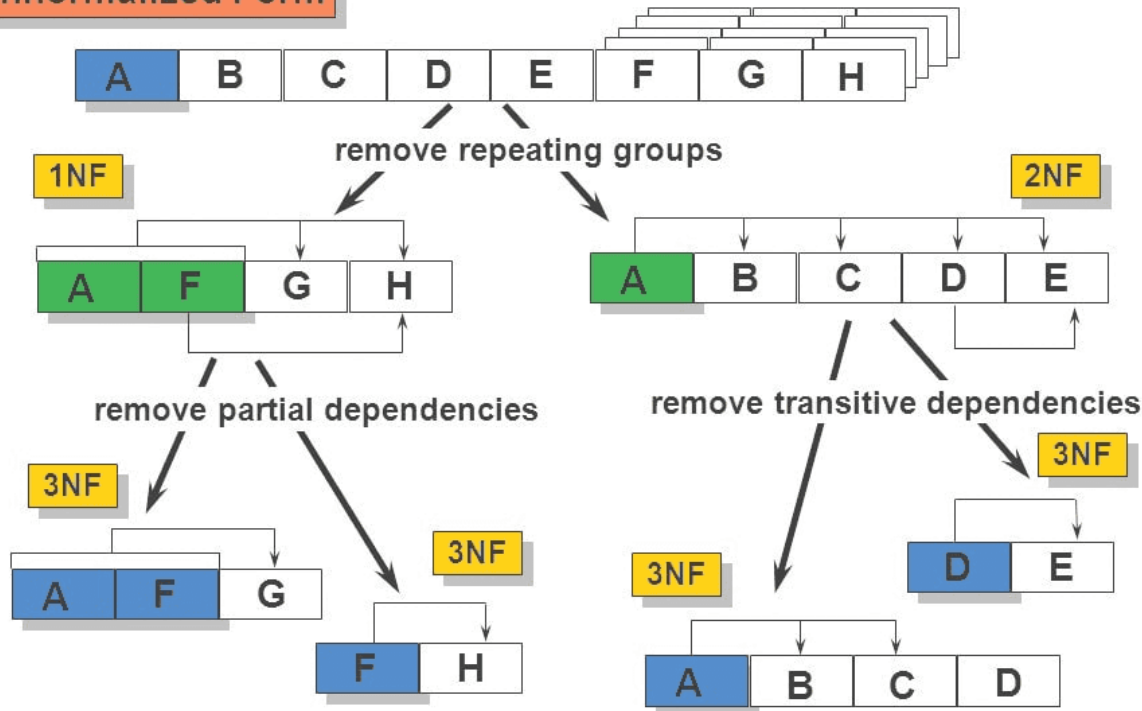
Normalization progresses from UNF → 1NF → 2NF → 3NF → BCNF

The previous diagram illustrates the process of database normalization, transitioning from an **Unnormalized Form (UNF)** to the **Third Normal Form (3NF)** and beyond. Each step ensures better organization and efficiency of the database.



Data Normalization Explained: Types, Examples, & Methods

Unnormalized Form



1. UNF:

- The table includes attributes **A, B, C, D, E, F, G, H**, where some attributes have repeating groups.

2. 1NF:

- Repeating groups (e.g., columns F, G, H) are removed.
- The data is split into two tables: one for attributes **A, F, G, H** and another for **A, B, C, D, E**.

3. 2NF:

- Partial dependencies are removed:
 - In **A, F, G, H**, split to isolate **A, F** and **G, H** based on dependencies.
 - In **A, B, C, D, E**, it is already in 2NF if **A** is the primary key.

3. 3NF:

- Transitive dependencies are removed:
 - In **A, B, C, D, E**, isolate **D, E** into a separate table because they depend on each other.
 - Final tables include:
 - A, F**
 - F, H**
 - A, B, C, D**
 - D, E**

RULE 1, RULE 2, RULE 3, RULE 4

Column 1	Column 2
A	X, Y
B	W, X
C	Y
D	Z

RULE 1

1. Each column should contain atomic values.
2. Entries like X,Y and W,X violate this rule.

DOB	Name
26-10-89	A
13-2-92	SK
16-11-65	SA
R	8-9-86

RULE 2

1. A Column should contain values that are of the same type
2. Do not inter-mix different types of values in any column

DOB	Name	Name
26-10-89	A	A
13-2-92	S	K
16-11-65	S	A
8-9-86	R	A

RULE 3

1. Each column should have a unique name
2. Same names leads to confusion at the time of data retrieval.

Roll_no	F_Name	L_Name
3	A	A
4	S	K
1	S	A
2	R	A

RULE 4

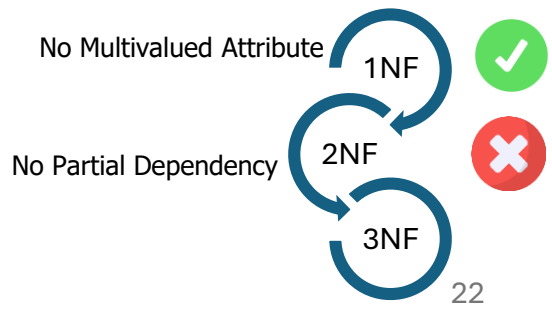
1. Order in which data is saved does not matter
2. Using SQL query., you can easily fetch data in any way order from a table.

How to achieve the 1st Normal Form?
You should have to follow these 4 rules for your table to be in 1st Normal Form.

Normalization Practice Exercise I | Third Normal Form | Denormalization

StudentID	CourseID	StudentName	Phone Number	CourseName	Grade	Teacher	TeacherEmail
101	CSE101	John Doe	1234567890	Introduction to CS	A	Dr. Smith	smith@university.edu
102	CSE101	Jane Smith	9876543210	Introduction to CS	B	Dr. Smith	smith@university.edu
101	MATH201	John Doe	1234567890	Calculus I	C	Prof. Johnson	johnson@university.edu
104	ENG301	Alice Brown	4567891230	Advanced English	A	Dr. Clark	clark@university.edu
102	CSE101	Jane Smith	9876543210	Introduction to CS	B	Dr. Smith	smith@university.edu
106	BIO101	Michael Green	7891234560	Biology Basics	A-	Dr. Wilson	wilson@university.edu
107	PHY101	Sarah Johnson	3216549870	Physics Principles	B+	Dr. Lewis	lewis@university.edu
104	ENG301	Alice Brown	4567891230	Advanced English	B	Dr. Clark	clark@university.edu
106	MATH201	Michael Green	7891234560	Calculus I	B	Prof. Johnson	johnson@university.edu
101	CSE101	John Doe	1234567890	Introduction to CS	A+	Dr. Smith	smith@university.edu

Identify functional dependencies in this table and normalize it to **Third Normal Form (3NF)**. Provide the resulting tables with primary and foreign keys clearly identified.



Demormalize Table

Normalization Practice Exercise I | Second Normal Form

Student Table

StudentID	StudentName	Phone Number
101	John Doe	1234567890
102	Jane Smith	9876543210
104	Alice Brown	4567891230
106	Michael Green	7891234560
107	Sarah Johnson	3216549870



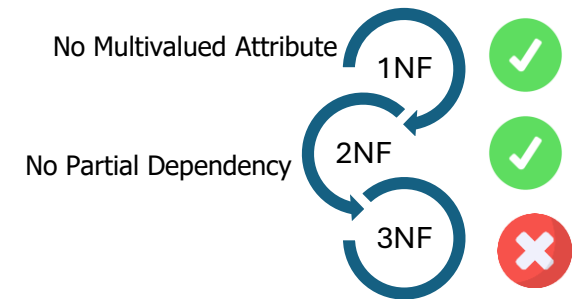
Course Table

CourseID	CourseName	Teacher	TeacherEmail
CSE101	Introduction to CS	Dr. Smith	smith@university.edu
MATH201	Calculus I	Prof. Johnson	johnson@university.edu
ENG301	Advanced English	Dr. Clark	clark@university.edu
BIO101	Biology Basics	Dr. Wilson	wilson@university.edu
PHY101	Physics Principles	Dr. Lewis	lewis@university.edu



Student-Course Table

StudentID	CourseID	Grade
101	CSE101	A
102	CSE101	B
101	MATH201	C
104	ENG301	A
102	CSE101	B
106	BIO101	A-
107	PHY101	B+
104	ENG301	B
106	MATH201	B
101	CSE101	A+



The partial dependency here we consider teacher's column as it depend to coursed.

Not in 3NF as there are transitive dependency like teacher.

NB: Our objectives is to remove Teacher from Course Table

Normalization Practice Exercise I | Third Normal Form

Student Table

StudentID	StudentName	Phone Number
101	John Doe	1234567890
102	Jane Smith	9876543210
104	Alice Brown	4567891230
106	Michael Green	7891234560
107	Sarah Johnson	3216549870

Student-Course Table

StudentID	CourseID	Grade
101	CSE101	A
102	CSE101	B
101	MATH201	C
104	ENG301	A
102	CSE101	B
106	BIO101	A-
107	PHY101	B+
104	ENG301	B
106	MATH201	B
101	CSE101	A+

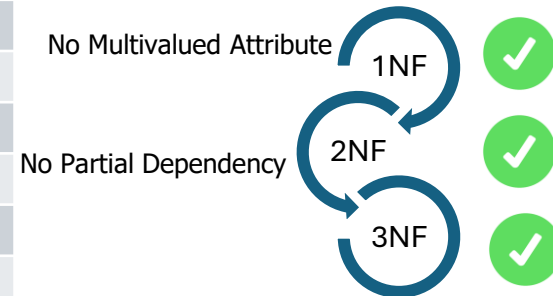


Course Table

CourseID	CourseName	TeacherID
CSE101	Introduction to CS	1
MATH201	Calculus I	2
ENG301	Advanced English	3
BIO101	Biology Basics	4
PHY101	Physics Principles	5

Teacher Table

TeacherID	Teacher	TeacherEmail
1	Dr. Smith	smith@university.edu
2	Prof. Johnson	johnson@university.edu
3	Dr. Clark	clark@university.edu
4	Dr. Wilson	wilson@university.edu
5	Dr. Lewis	lewis@university.edu



Verify 3NF

- Each table has a primary key.
- Should be in the 2nd NF
- All non-key attributes depend only on the primary key (no partial or transitive dependencies).

Scenario: College Enrollment Table | 3.5 NF or Boyce-Codd Normal Form (BCNF) (1/4)

Step 1: Denormalized Table (Violating 1NF)

We start with a **denormalized table** containing repeating groups, which violates **3NF** due to **transitive dependencies**. The goal is to normalize it through 1NF, 2NF, 3NF, and finally **BCNF** (also referred to as **3.5NF** in some contexts).

Student_ID	Subject	Professor
101	PLSQL, Big Data	Dr. Smith
101	RDBMS	Dr. Johnson
101	Java	Dr. Lee
102	Big Data	Dr. Smith
102	PLSQL	Dr. Johnson
103	Java	Dr. Brown
104	RDBMS	Dr. Lee

Step 2: First Normal Form (1NF)



To bring the table into **1NF**, we eliminate **repeating groups** and ensure that each record contains atomic values (no multiple values in a single cell). In this case, we simply separate each subject and professor combination into its own row.

Student_ID	Subject	Professor
101	PLSQL	Dr. Smith
101	RDBMS	Dr. Johnson
101	Java	Dr. Lee
102	Big Data	Dr. Smith
102	PLSQL	Dr. Johnson
103	Java	Dr. Brown
104	RDBMS	Dr. Lee

Now the table is in **1NF**, as there are no repeating groups, and each column holds only atomic values.

Scenario: College Enrollment Table | 3.5 NF or Boyce-Codd Normal Form (BCNF) (2/4)

Step 3: Second Normal Form (2NF)



To bring the table into **2NF**, we need to eliminate partial dependencies, which occur when a non-key attribute depends on only a part of the composite key. The **composite key** here is a combination of **Student_ID** and **Subject**, because both are needed to uniquely identify each record.

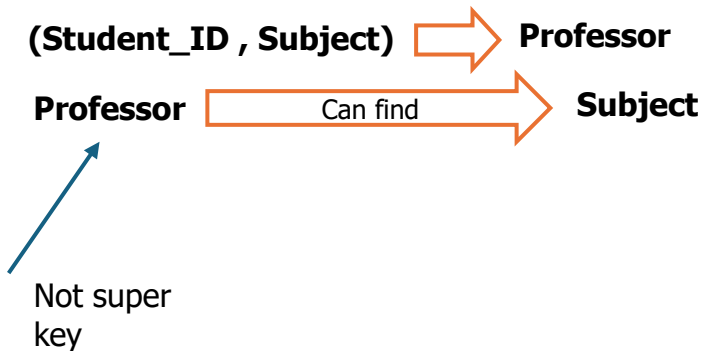
- **Professor** depends on **Subject**, but not on the full composite key (**Student_ID** + **Subject**).
- So, we need to split the table into two:

Students Subjects Table (2NF)

Student_ID	Subject
101	PLSQL
101	RDBMS
101	Java
102	Big Data
102	PLSQL
103	Java
104	RDBMS

Subjects Professors Table (2NF)

Subject	Professor
PLSQL	Dr. Smith
RDBMS	Dr. Johnson
Java	Dr. Lee
Big Data	Dr. Smith
RDBMS	Dr. Lee
Java	Dr. Brown



In this step, we've eliminated partial dependency by splitting the tables such that **Professor** now depends only on **Subject**, and **Student_ID** is now associated only with the **Subject** in the **Students Subjects Table**.

Scenario: College Enrollment Table | 3.5 NF or Boyce-Codd Normal Form (BCNF) (3/4)

Step 4: Third Normal Form (3NF)



To bring the table into **3NF**, we must eliminate **transitive dependencies**, where a non-key attribute depends on another non-key attribute.

In this case:

- **Professor** depends on **Subject** (i.e., a subject has one professor).
- But **Student_ID** depends directly on **Subject**, and indirectly on **Professor**.

So, we can split the tables further:

Students Table (3NF)

Student_ID	Subject
101	PLSQL
101	RDBMS
101	Java
102	Big Data
102	PLSQL
103	Java
104	RDBMS

Subjects Table (3NF)

Subject	Professor
PLSQL	Dr. Smith
RDBMS	Dr. Johnson
Java	Dr. Lee
Big Data	Dr. Smith

Here, **Subjects Table** contains unique subjects with their corresponding professors, and **Students Table** just links students with subjects. There is no transitive dependency anymore, as **Professor** directly depends on **Subject**, not indirectly through **Student_ID**.

Scenario: College Enrollment Table | 3.5 NF or Boyce-Codd Normal Form (BCNF) (4/4)

Step 5: Boyce-Codd Normal Form (BCNF or 3.5NF)



To bring the table to **BCNF** (also referred to as **3.5NF**), we need to ensure that for every functional dependency, the left-hand side (determinant) is a **superkey**.

In the **Subjects Table**:

- Subject → Professor (But **Subject** is not a superkey in this case because a subject can have only one professor, but **Professor** is not uniquely identifying a record.)

To resolve this, we split the tables further.

Students Table (BCNF)

Student_ID	Subject
101	PLSQL
101	RDBMS
101	Java
102	Big Data
102	PLSQL
103	Java
104	RDBMS

Professors Table (BCNF)

Professor_ID	Professor
1	Dr. Smith
2	Dr. Johnson
3	Dr. Lee
4	Dr. Brown

Subjects Table (BCNF)

Subject	Professor_ID
PLSQL	1
RDBMS	2
Java	3
Big Data	1

Conclusion

- 1NF:** We ensured that each field contains atomic values.
- 2NF:** We removed partial dependencies.
- 3NF:** We removed transitive dependencies.
- 4. BCNF (3.5NF):** We ensured that every determinant is a superkey, thus eliminating any remaining redundancies.

This normalized schema ensures consistency, avoids data anomalies, and adheres to **Boyce-Codd Normal Form** (BCNF or 3.5NF).

Let me know if you need further clarification!

Boyce-Codd Normal Form (BCNF)

Boyce-Codd Normal Form (BCNF) is an advanced version of the **Third Normal Form (3NF)** used in database normalization. It ensures a higher level of data integrity by addressing anomalies that 3NF might not handle effectively.

BCNF Rules:

- The table must first satisfy **3NF**.
- For every **functional dependency** ($A \rightarrow B$), **A** must be a **superkey** (i.e., it should uniquely identify all rows in the table).

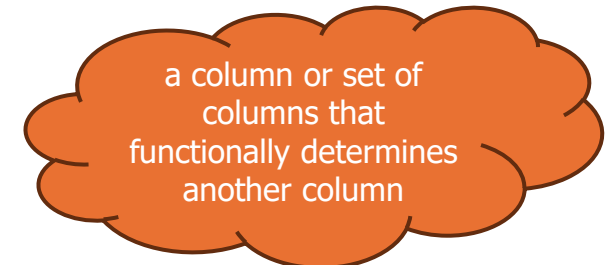
When is BCNF Needed?

BCNF is required when a table satisfies 3NF but still has anomalies due to **overlapping candidate keys** or **dependencies between non-prime attributes**.



Summary Table

Normal Form	Key Concept	Goal
1NF	Eliminate repeating groups and multivalued cells.	Each cell contains atomic values.
2NF	Eliminate partial dependencies.	Every non-key column depends on the whole primary key.
3NF	Eliminate transitive dependencies.	No non-key column depends on another non-key column.
BCNF	Eliminate all anomalies.	Every determinant is a superkey.



Understanding Relationship Types Between Tables in Database

Relationship Type	Description	Real-Life Example
One-to-One (1:1)	Each record in one table corresponds to one record in another table.	- Patients and their medical details in a hospital database. - Employees and salary details in a firm.
One-to-Many (1:N)	One record in a parent table relates to multiple records in a child table.	- Customers and their multiple orders in an e-commerce system. - Teachers assigned to multiple students.
Many-to-Many (M:N)	Multiple records in one table relate to multiple records in another table.	- Students enrolling in multiple courses, and courses having many students. - Members borrowing books.
Self-Referencing	Records in a table reference other records in the same table.	- Employees reporting to other employees (managers) within a company. - Product categories and subcategories.

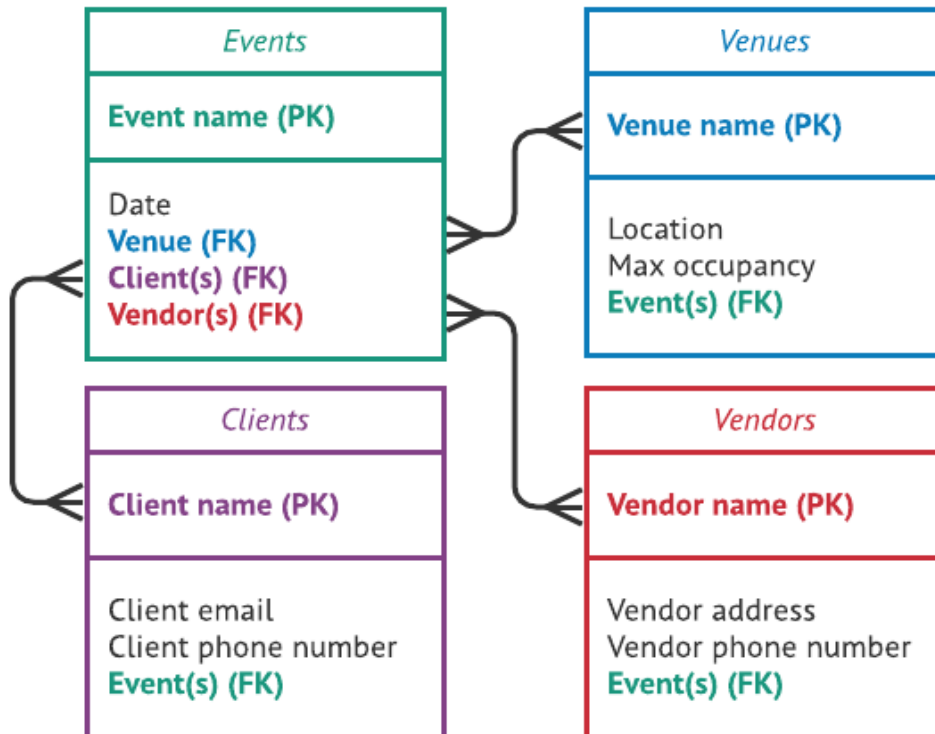
- **Relationships** help organize and connect data across multiple tables, improving database normalization.
- **Primary keys** and **foreign keys** ensure data integrity and enforce constraints.
- Proper design of **relationships** enhances scalability, flexibility, and performance in relational databases.

These relationship types form the foundation of relational database systems like Oracle, enabling efficient data management and retrieval.

- **Popular Relational Databases:** Common RDBMS include:



Key Components in Databases with Multiple Tables



Relationships:

- **Events and Venues:** Each event takes place at one venue, and a venue can host multiple events (one-to-many).
- **Events and Clients:** Each event is organized by one or more clients, and a client can organize multiple events (many-to-many).
- **Events and Vendors:** Each event may involve multiple vendors, and each vendor can participate in multiple events (many-to-many).

Summary:

This design allows for efficient data management, facilitating queries about events, their locations, clients, and associated vendors. By utilizing foreign keys, it establishes relationships that maintain data integrity and enable relational querying across these interconnected tables.

Relational databases offer a robust and efficient way to manage structured data, providing fast, reliable access through powerful indexing and relationship management features.

Relational Database relationships (1:1)

1. One-to-One Relationship



Definition: Each record in Table A is linked to only one record in Table B, and vice versa.

- **Example:** A person's passport is linked to one unique individual, or a person's home address may be tied to a specific ZIP code.
- **Use Case:** One-to-one relationships are often used to separate sensitive data for security purposes or to limit access. For example, separating a patient's contact information from their medical history allows different levels of access for administrative and medical staff.

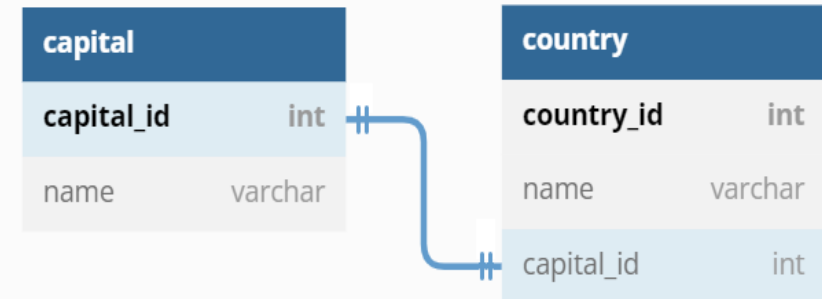
Key Points:

- Each country has only one capital city, and each capital city corresponds to exactly one country.
- The one-to-one relationship ensures that there is a direct, unique association between the two tables.

How It Works:

- Each CountryID in the **Capitals Table** uniquely links a capital to a specific country.
- The CountryID in the Capitals table acts as a **Foreign Key** that references the **Primary Key** of the Countries table.
- For example, France (CountryID = 1) is linked to Paris, and Germany (CountryID = 2) is linked to Berlin.

One to one database relationship



Source Image: <https://phoenixnap.com/kb/database-relationships>

Relational Database relationships (1:N)

2. One-to-Many Relationship

Definition: A single record in Table A can be associated with multiple records in Table B, but each record in Table B is linked to only one record in Table A.



- **Example:**

- A customer (Table A) may place many orders (Table B), but each order belongs to only one customer.
- A book (Table A) can have multiple authors (Table B), but each author is linked to one book in that context.

- **Use Case:** This is the most common relationship type and is used to represent hierarchical data or parent-child relationships, such as customers and orders, products and categories, or cities and ZIP codes.

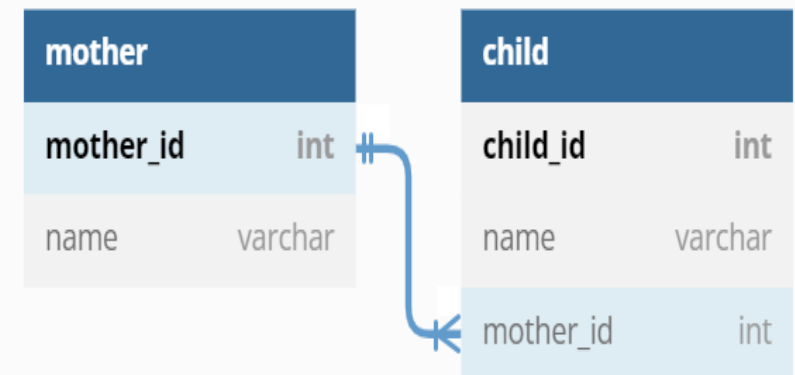
Key Points:

- The **Primary Key** (MotherID) uniquely identifies each record in the Mothers table.
- The **Foreign Key** (MotherID in the Children table) establishes a link to the Mothers table.
- Multiple records in the Children table can have the same MotherID, meaning multiple children can belong to the same mother.

How It Works:

- The **MotherID** column in the Children table acts as the foreign key that connects each child to their respective mother.
- Even though multiple children can have the same MotherID, each child is associated with only one mother, illustrating the one-to-many relationship.

One to many database relationship



Relational Database relationships (N:N)

3. Many-to-Many Relationship

Definition: Records in Table A can relate to multiple records in Table B, and records in Table B can relate to multiple records in Table A.



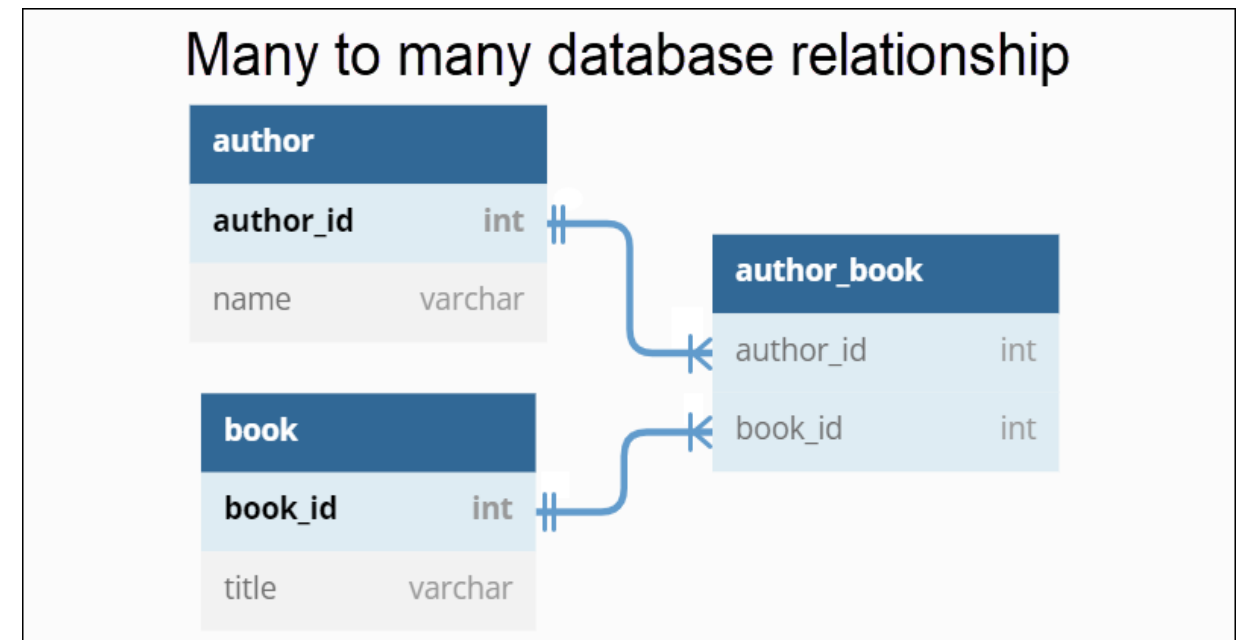
- **Example:**

- A student (Table A) can enroll in many courses (Table B), and each course can have many students.
- A book (Table A) can belong to many categories (Table B), and each category can include multiple books.

- **Use Case:** To manage many-to-many relationships, you need an **intermediate table** (often called a junction or linking table) to connect the two tables. For example, a "StudentCourses" table could link students to the courses they are enrolled in.

Key Points:

- **Junction Table** (BookAuthors) is essential to establish and manage the many-to-many relationship.
- **Books:** A single book can have multiple authors.
- **Authors:** A single author can write multiple books.



Relational Database relationships (Self Referencing)

4. Self Referencing

Definition: Is a type of relationship in a database where a table has a relationship with itself. This means a column in the table refers back to another column in the same table, typically through a foreign key.

Example:

- E-commerce platforms for product navigation.
- File systems for folder hierarchies.
- Taxonomy for blog or article categorization.

Use Case:

- Track relationships within a family or genealogy.
- Model nested departmental structures (e.g., Head Office → IT → Software Development).

Key Points:

The diagram represents a **self-referencing relationship** in the Employee table:

Table Structure:

- employee_id: Primary key for each employee.
- firstname and lastname: Employee's name.
- manager_id: Foreign key referencing employee_id within the same table.

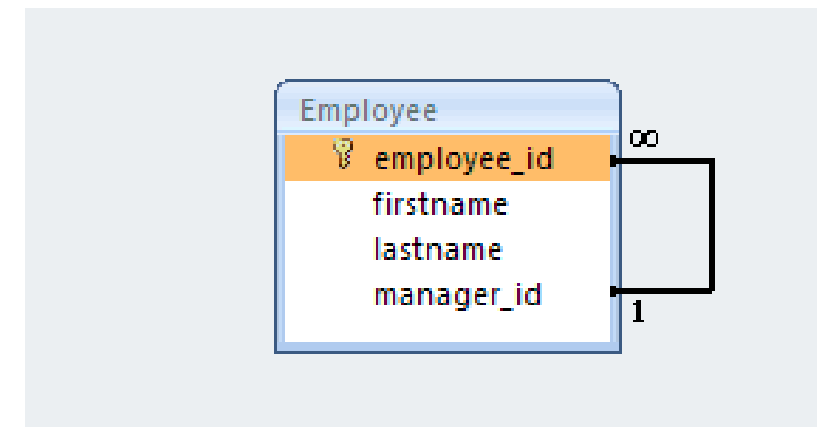
Relationship:

- One manager (employee_id) can supervise multiple employees (manager_id).
- An employee's manager_id refers to their manager's employee_id.

Special Case:

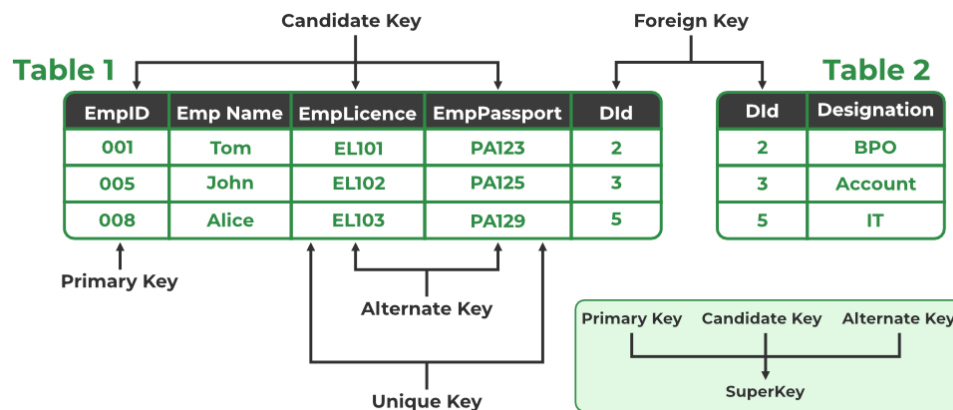
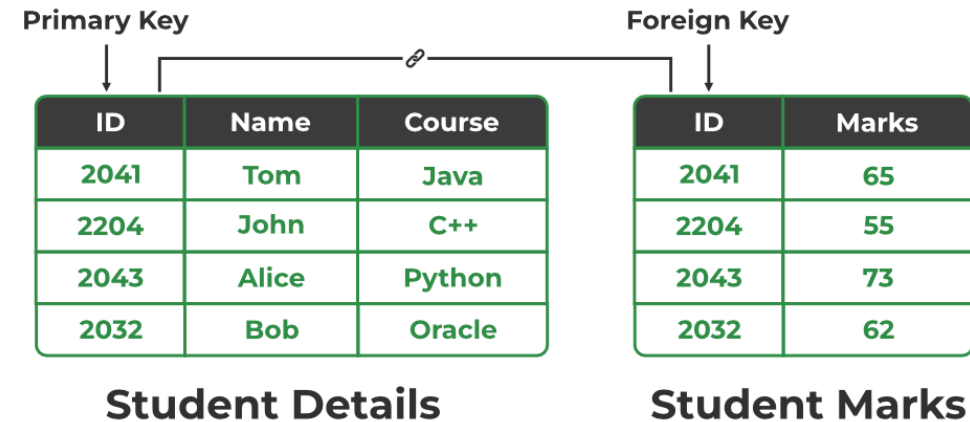
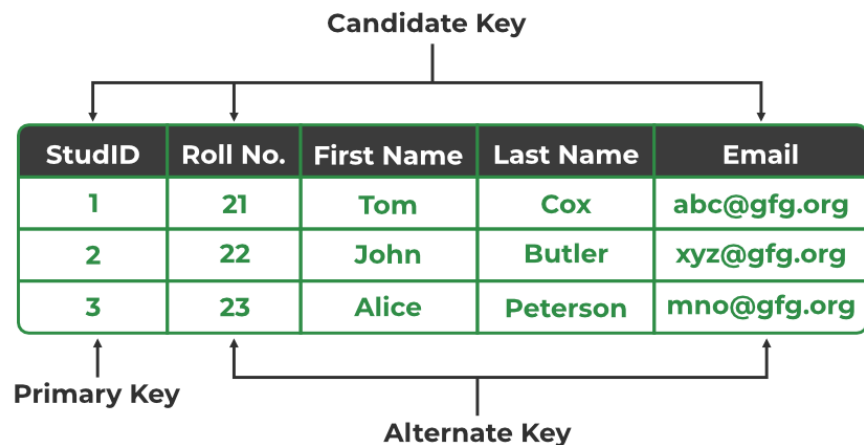
- manager_id = NULL indicates an employee without a manager (e.g., CEO).

This design models hierarchical structures, such as employee-manager relationships, in a single table.



Source Image: <https://www.viralpatel.net/hibernate-self-join-a-one-to-many-mapping/>

Types of Keys in Relational Model (Candidate, Super, Primary, Alternate and Foreign)



- **Candidate Key:** A set of one or more columns that can uniquely identify a row in a table. Every table can have multiple candidate keys, but one is chosen as the primary key.
- **Super Key:** A set of columns that uniquely identify a row in a table. A super key may include additional attributes not necessary for unique identification, unlike a candidate key.
Example: In **Table-1**, Primary key, Unique key, Alternate key are a subset of Super Keys.
- **Alternate Key:** Any candidate key that is not chosen as the primary key. Alternate keys are unique and serve as backup identifiers.
- **Composite Key:** A primary or candidate key that consists of two or more columns combined to uniquely identify a row in a table.

Constraints and Data Integrity



1. Unique Constraint:

- This constraint ensures that all values in a specific column (or combination of columns) are unique across the table, preventing duplicate values.

Example:

```
CREATE TABLE Users ( UserID INT PRIMARY KEY, Email VARCHAR(100) UNIQUE );
```

2. Not Null:

- A **NOT NULL** constraint prevents null values from being inserted into a column, ensuring that every record has a valid entry in that field.

Example:

```
CREATE TABLE Products ( ProductID INT PRIMARY KEY, ProductName VARCHAR(100) NOT NULL );
```

3. Check Constraint:

- This constraint enforces a specific condition that the data in a column must meet. If the condition is not satisfied, the database will reject the entry.

Example:

```
CREATE TABLE Employees ( EmployeeID INT PRIMARY KEY, Age INT, CHECK (Age >= 18) );
```

Creating and Modifying Constraints

Defining Constraints During Table Creation:



Constraints can be defined at the time of table creation, as shown in the examples above. By specifying constraints at this stage, you ensure that only valid data is allowed into the table from the start.

1. Adding Constraints to Existing Tables:

- Constraints can also be added to an existing table using the ALTER TABLE statement.
- Example:

```
ALTER TABLE Employees ADD CONSTRAINT fk_department FOREIGN KEY (DepartmentID) REFERENCES Departments(DepartmentID);
```

2. Enforcing and Disabling Constraints:

- Constraints can be enforced to maintain integrity or temporarily disabled when necessary (e.g., during bulk inserts).
- Example of disabling a constraint:

```
ALTER TABLE Orders NOCHECK CONSTRAINT fk_customer;
```

- Example of re-enabling a constraint:

```
ALTER TABLE Orders CHECK CONSTRAINT fk_customer;
```

SQL Joins

Objective: Teach how to combine data from multiple tables, use subqueries, and apply aggregate functions.



1. SELECT Statement:

- The SELECT statement is used to retrieve data from one or more tables in a database. It allows you to specify which columns you want to retrieve.
- Using subqueries within SELECT, INSERT, UPDATE, and DELETE.
- **Example:** *SELECT FirstName, LastName FROM Customers;*

2. WHERE Clause:

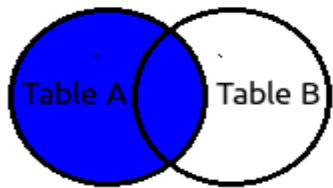
- The WHERE clause is used to filter rows in a query based on specified conditions, allowing you to retrieve only the records that meet those criteria.
- **Example:** *SELECT * FROM Orders WHERE OrderDate = '2024-08-30';*

3. JOIN:

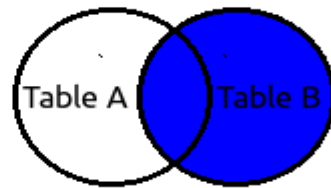
A JOIN combines rows from two or more tables based on a related column. This allows you to link related data stored in different tables.

- **INNER JOIN**
Combines rows from two or more tables and returns only the rows with matching values in the specified columns.
- **LEFT JOIN (LEFT OUTER JOIN)**
Returns all rows from the left table and the matching rows from the right table. Non-matching rows in the right table are filled with NULL.
- **RIGHT JOIN (RIGHT OUTER JOIN)**
Returns all rows from the right table and the matching rows from the left table. Non-matching rows in the left table are filled with NULL.
- **FULL JOIN (FULL OUTER JOIN)**
Returns all rows from both tables. Non-matching rows in either table are filled with NULL.
- **CROSS JOIN**
Produces the Cartesian product of two tables, pairing each row from the first table with every row from the second table.
- **UNION**
Combines the results of two or more SELECT queries into a single result set. Duplicate rows are removed by default unless UNION ALL is used.
- **UNION ALL**
Returns all records, including duplicates. It's often used in reporting and analysis.

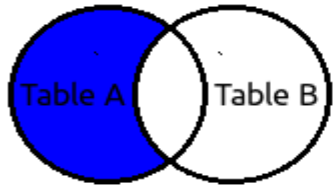
SQL Joins | Inner | Outer | Right | Left | Cross Join



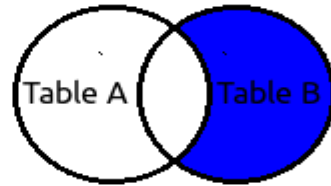
SELECT [list] FROM
[Table A] A
LEFT JOIN
[Table B] B
ON A.Value = B.Value



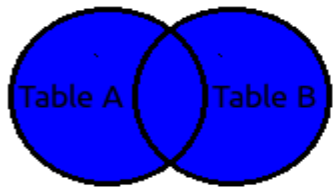
SELECT [list] FROM
[Table A] A
RIGHT JOIN
[Table B] B
ON A.Value = B.Value



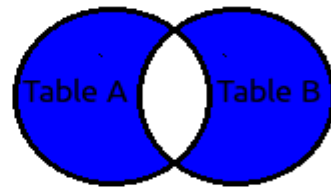
SELECT [list] FROM
[Table A] A
LEFT JOIN
[Table B] B
ON A.Value = B.Value
WHERE B.Value IS NULL



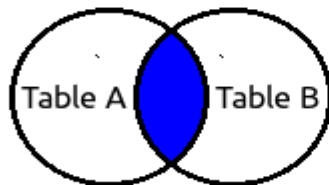
SELECT [list] FROM
[Table A] A
RIGHT JOIN
[Table B] B
ON A.Value = B.Value
WHERE A.Value IS NULL



SELECT [list] FROM
[Table A] A
FULL OUTER JOIN
[Table B] B
ON A.Value = B.Value

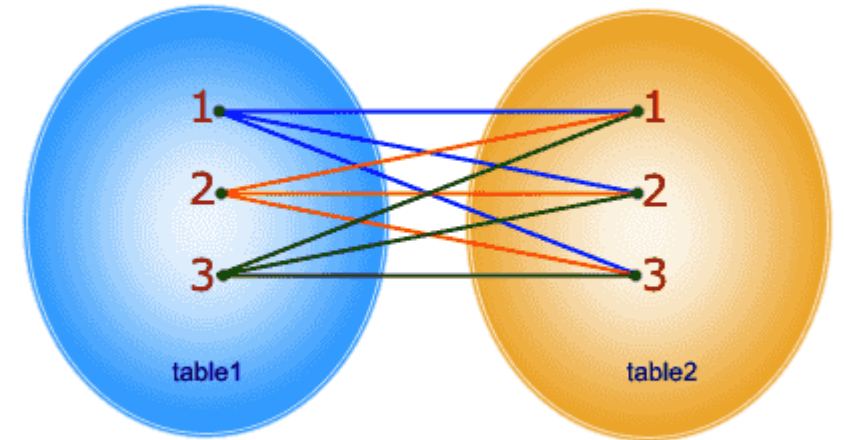


SELECT [list] FROM
[Table A] A
FULL OUTER JOIN
[Table B] B
ON A.Value = B.Value
WHERE A.Value IS NULL
OR B.Value IS NULL



SELECT [list] FROM
[Table A] A
INNER JOIN
[Table B] B
ON A.Value = B.Value

SELECT * FROM table1 CROSS JOIN table2;

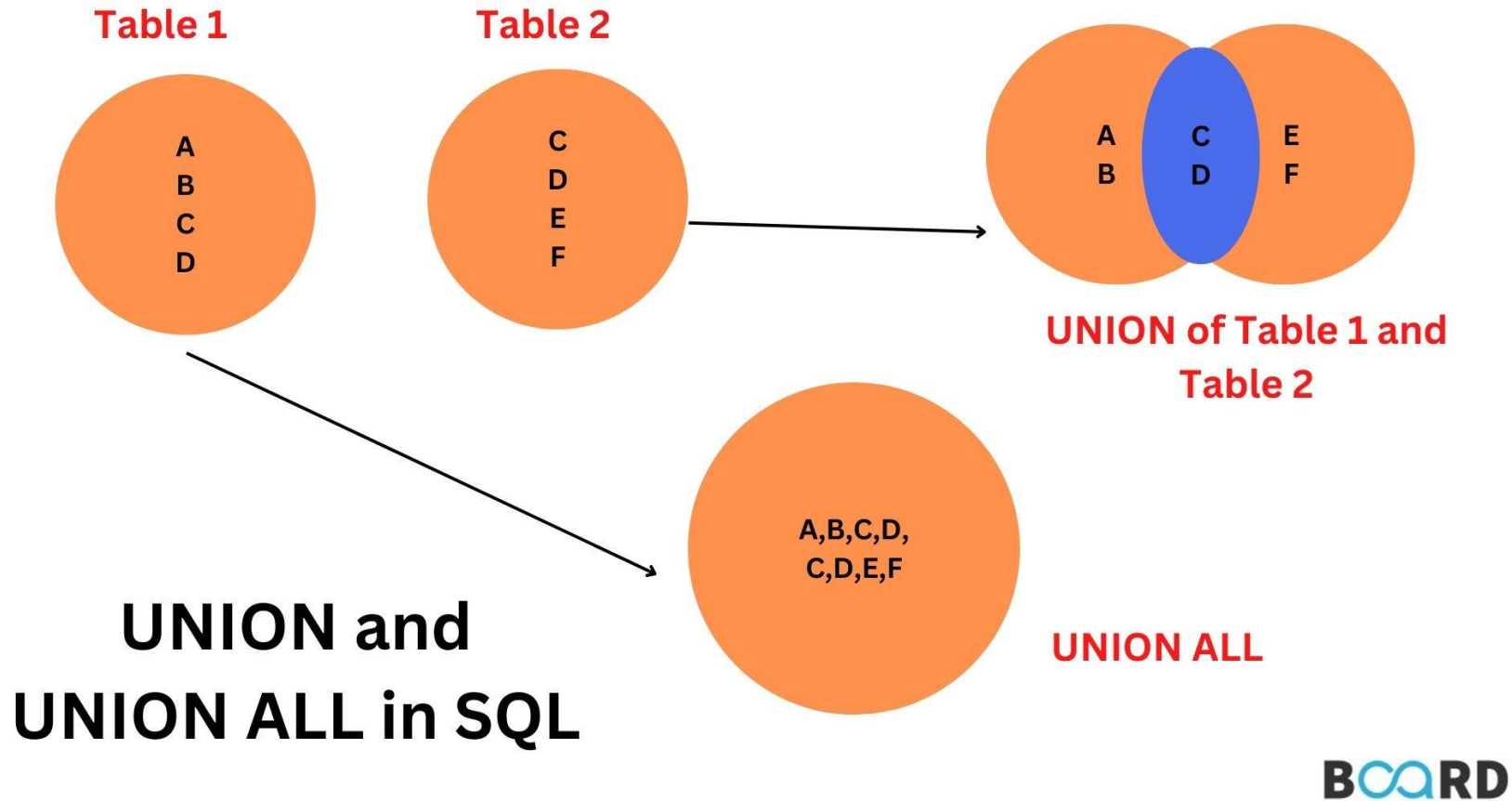


In CROSS JOIN, each row from 1st table joins with all the rows of another table.
If 1st table contain x rows and y rows in 2nd one the result set will be x * y rows.

Source Image: <https://www.w3resource.com/sql/joins/cross-join.php>

Source Image: <https://www.geekphilip.com/2012/04/01/visual-explanation-of-sql-joins/>

SQL Joins - Union



Example of SQL Joins

Objective: Teach how to combine data from multiple tables, use subqueries, and apply aggregate functions.



Customer_ID	Customer_Name	Email	Phone
1	John Doe	john.doe@gmail.com	123-456-7890
2	Jane Smith	jane.smith@gmail.com	987-654-3210
3	Bob Johnson	bob.johnson@gmail.com	456-789-0123
4	Patrick Kaka	Null	Null

Event_ID	Event_Name	Event_Date	Location	Customer_Name	Email	Phone
101	Wedding Ceremony	2024-05-20	Grand Hall	John Doe	john.doe@gmail.com	123-456-7890
102	Corporate Meeting	2024-06-15	Conference Room	Jane Smith	jane.smith@gmail.com	987-654-3210
103	Birthday Party	2024-07-10	Banquet Hall	Bob Johnson	bob.johnson@gmail.com	456-789-0123

Solution Left & Right Join

Left Join

```
SELECT c.Customer_ID, c.Customer_Name, c.Email AS Customer_Email,  
e.Event_ID, e.Event_Name  
FROM Customers c  
LEFT JOIN Events e  
ON c.Customer_Name = e.Customer_Name;
```

Customer_ID	Customer_Name	Customer_Email	Event_ID	Event_Name
1	John Doe	john.doe@gmail.com	101	Wedding Ceremony
2	Jane Smith	jane.smith@gmail.com	102	Corporate Meeting
3	Bob Johnson	bob.johnson@gmail.com	103	Birthday Party
4	Patrick Kaka	NULL	NULL	NULL

Right Join

```
SELECT c.Customer_ID, c.Customer_Name, e.Event_ID, e.Event_Name  
FROM Customers c  
RIGHT JOIN Events e  
ON c.Customer_Name = e.Customer_Name;
```

Customer_ID	Customer_Name	Event_ID	Event_Name
1	John Doe	101	Wedding Ceremony
2	Jane Smith	102	Corporate Meeting
3	Bob Johnson	103	Birthday Party

Solution Left & Right Join WHERE B or A IS NULL |

Left Join Where B is Null

```
SELECT c.Customer_ID, c.Customer_Name, c.Email  
FROM Customers c  
LEFT JOIN Events e  
ON c.Customer_Name = e.Customer_Name  
WHERE e.Event_ID IS NULL;
```

Customer_ID	Customer_Name	Email
4	Patrick Kaka	NULL



Right Join Where A is Null

```
SELECT e.Event_ID, e.Event_Name, e.Location  
FROM Customers c  
RIGHT JOIN Events e  
ON c.Customer_Name = e.Customer_Name  
WHERE c.Customer_ID IS NULL;
```

Event_ID	Event_Name	Location

Result:

No results since all events have associated customers.

Solution Full Outer Join | Full Outer Join WHERE B IS NULL OR A IS NULL

Full Outer Join

```
SELECT c.Customer_ID, c.Customer_Name, e.Event_ID, e.Event_Name
FROM Customers c
FULL OUTER JOIN Events e
ON c.Customer_Name = e.Customer_Name;
```

Customer_ID	Customer_Name	Event_ID	Event_Name
1	John Doe	101	Wedding Ceremony
2	Jane Smith	102	Corporate Meeting
3	Bob Johnson	103	Birthday Party
4	Patrick Kaka	NULL	NULL

Full Outer Join WHERE B IS NULL OR A IS NULL

```
SELECT c.Customer_ID, c.Customer_Name, e.Event_ID, e.Event_Name
FROM Customers c
FULL OUTER JOIN Events e
ON c.Customer_Name = e.Customer_Name
WHERE e.Event_ID IS NULL OR c.Customer_ID IS NULL;
```

Customer_ID	Customer_Name	Event_ID	Event_Name
4	Patrick Kaka	NULL	NULL

Solution Cross Join

```
SELECT c.Customer_ID, c.Customer_Name, e.Event_ID,  
       e.Event_Name  
FROM Customers c  
CROSS JOIN Events e;
```

Customer_ID	Customer_Name	Event_ID	Event_Name
1	John Doe	101	Wedding Ceremony
1	John Doe	102	Corporate Meeting
1	John Doe	103	Birthday Party
2	Jane Smith	101	Wedding Ceremony
2	Jane Smith	102	Corporate Meeting
2	Jane Smith	103	Birthday Party
3	Bob Johnson	101	Wedding Ceremony
3	Bob Johnson	102	Corporate Meeting
3	Bob Johnson	103	Birthday Party
4	Patrick Kaka	101	Wedding Ceremony
4	Patrick Kaka	102	Corporate Meeting
4	Patrick Kaka	103	Birthday Party

Use Cases:

- Cross joins are rarely used in practice unless explicitly needed, as the resulting dataset grows exponentially.
- They are useful for testing or when calculating combinations of data (e.g., comparing all customers with all events).

Solution Union | Union ALL

Union

```
SELECT Customer_Name AS Name
FROM Customers
UNION
SELECT Event_Name AS Name
FROM Events;
```

Name

John Doe

Jane Smith

Bob Johnson

Patrick Kaka

Wedding Ceremony

Corporate Meeting

Birthday Party

Explanation:

- The **UNION** operator merges the two queries.
- Duplicates are removed from the combined result set.

Union All

```
SELECT Customer_Name AS Name
FROM Customers
UNION ALL
SELECT Event_Name AS Name
FROM Events;
```

Name

John Doe

Jane Smith

Bob Johnson

Patrick Kaka

Wedding Ceremony

Corporate Meeting

Birthday Party

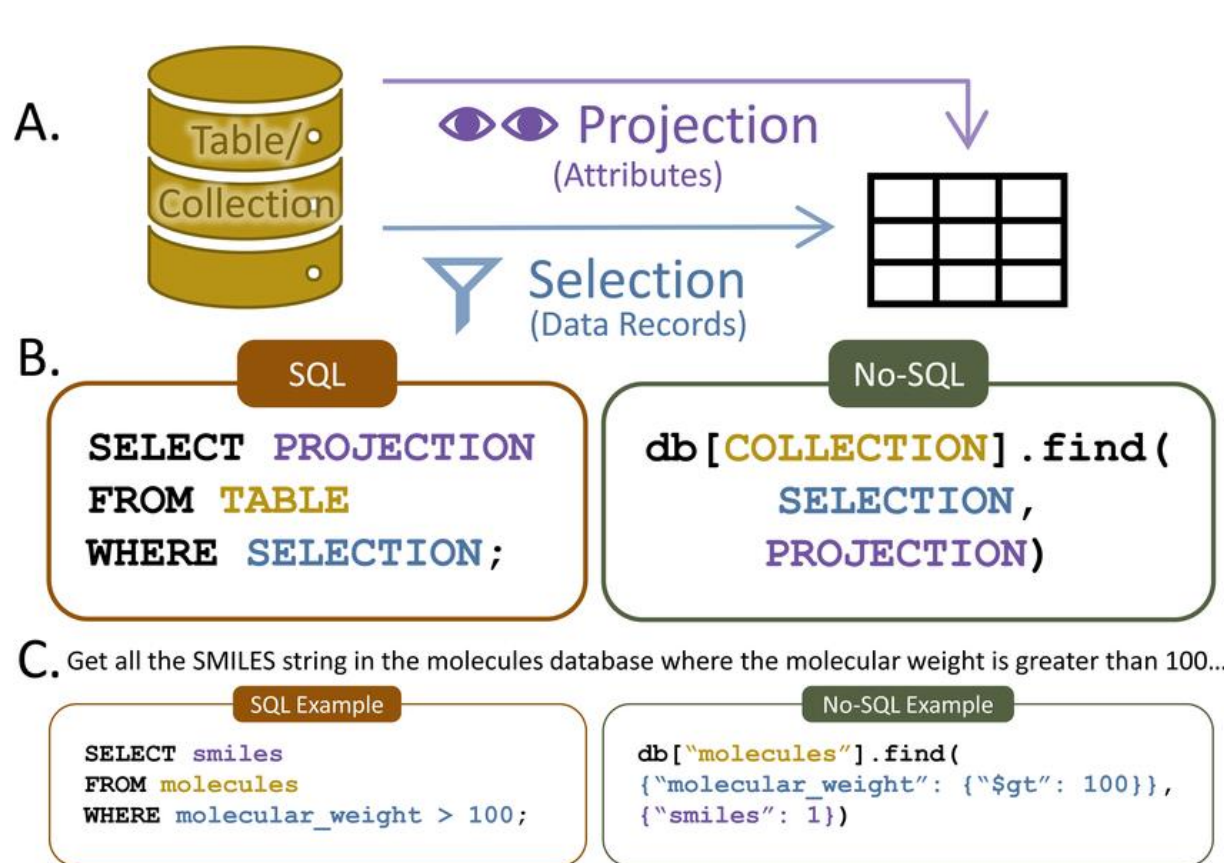
Use Cases for UNION:

1. Combine datasets from multiple sources.
2. Aggregate different types of entities into a single list (e.g., customers and event names).
3. Retrieve distinct values across datasets.

Important Notes:

- If column names differ in the two queries, you must explicitly alias them to provide consistency.
- The **ORDER BY** clause can only be used at the end of the **UNION** query and applies to the entire result set.

Depiction of the SQL selection , projection



In SQL, **selection** and **projection** are fundamental operations used to retrieve and manipulate data from a database. They are part of the relational algebra that underlies SQL query execution. Here's what they mean:

1. Selection (WHERE clause):

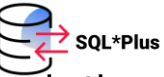
- **Definition:** Selection refers to **filtering rows** from a table based on specified conditions. It retrieves only those rows that meet the given criteria.
- **How it works:** The selection operation uses the WHERE clause to specify the condition for filtering.
- **Example:** Selecting employees from the Employees table whose salary is greater than 50,000:

```
SELECT * FROM Employees  
WHERE Salary > 50000;
```

In this example, only rows (employees) where the salary is greater than 50,000 will be selected.

Depiction of the SQL selection , projection

2. Projection (SELECT clause):



- **Definition:** Projection refers to selecting **specific columns** (attributes) from a table. Instead of retrieving all columns, you can retrieve only those that are relevant.
- **How it works:** The projection operation is performed using the SELECT clause, where you specify the column names you want to display.
- **Example:** Projecting (selecting) the EmployeeID and Name columns from the Employees table:

SELECT EmployeeID, Name FROM Employees;

In this case, only the EmployeeID and Name columns will be returned, while the other columns (e.g., Salary, DepartmentID) will be ignored.

Combining Selection and Projection:

You can combine both selection and projection in a single query, as shown below:

SELECT EmployeeID, Name FROM Employees WHERE Salary > 50000;

In this example, you are **projecting** (selecting) only the EmployeeID and Name columns and **filtering** the rows where Salary is greater than 50,000 (selection).

Key Points:

- **Selection** (WHERE) reduces the number of rows by filtering based on conditions.
- **Projection** (SELECT) reduces the number of columns by choosing only the relevant ones.

Window Function

1. **SQL window functions** are essential for advanced **data analysis** and **database management**.
2. **A window function in SQL** is a type of function that **performs a calculation across a set of rows** related to the current row. **Unlike aggregate functions, window functions do not group rows together**; instead, they allow you to **retain individual row data** while performing the calculation.

Aggregate Functions



- ☐ **AVG():** Calculates the average value in the window.
- ☐ **MAX():** Finds the maximum value in the window.
- ☐ **MIN():** Finds the minimum value in the window.
- ☐ **SUM():** Calculates the total sum of values in the window.
- ☐ **COUNT():** Counts the number of rows in the window.

Ranking Functions



- ☐ **ROW_NUMBER():** Assigns a unique sequential number to each row in the window.
- ☐ **RANK():** Assigns a rank to each row, with gaps for ties.
- ☐ **DENSE_RANK():** Assigns a rank to each row without gaps for ties.
- ☐ **PERCENT_RANK():** Computes the relative rank of a row as a percentage.
- ☐ **NTILE():** Divides rows into a specified number of groups and assigns a group number.

Value Access Functions



- ☐ **LAG():** Accesses a value from a previous row in the window.
- ☐ **LEAD():** Accesses a value from a subsequent row in the window.
- ☐ **FIRST_VALUE():** Returns the first value in the window.
- ☐ **LAST_VALUE():** Returns the last value in the window.
- ☐ **NTH_VALUE():** Returns the nth value in the window.



They are called **window functions** because they operate over a "**window**" or a specific set of rows in a result set, rather than the entire dataset.

Window Function Definitions

Window Function	Type	Definition	Example
ROW_NUMBER()	Ranking	Assigns a unique row number starting at 1 for each row in the result set.	<i>SELECT ROW_NUMBER() OVER (PARTITION BY column_name ORDER BY another_column) AS row_num, column_name FROM table_name;</i>
RANK()	Ranking	Assigns a rank to each row, but skips ranks when there are ties.	<i>SELECT RANK() OVER (PARTITION BY column_name ORDER BY another_column) AS rank, column_name FROM table_name;</i>
DENSE_RANK()	Ranking	Similar to RANK(), but no ranks are skipped when there are ties.	<i>SELECT DENSE_RANK() OVER (PARTITION BY column_name ORDER BY another_column) AS dense_rank, column_name FROM table_name;</i>
NTILE(n)	Distribution	Divides the result set into n approximately equal parts and assigns each row a bucket number.	<i>SELECT NTILE(4) OVER (PARTITION BY column_name ORDER BY another_column) AS quartile, column_name FROM table_name;</i>
SUM()	Aggregation	Calculates the sum of values over a specified range of rows.	<i>SELECT column_name, SUM(column_name) OVER (PARTITION BY another_column ORDER BY yet_another_column) AS total_sum FROM table_name;</i>
AVG()	Aggregation	Calculates the average value over a set of rows.	<i>SELECT column_name, AVG(column_name) OVER (PARTITION BY another_column ORDER BY yet_another_column) AS average_value FROM table_name;</i>
LEAD()	Value Access	Provides access to the next row's value in the result set.	<i>SELECT column_name, LEAD(column_name) OVER (PARTITION BY another_column ORDER BY yet_another_column) AS next_value FROM table_name;</i>
LAG()	Value Access	Provides access to the previous row's value in the result set.	<i>SELECT column_name, LAG(column_name) OVER (PARTITION BY another_column ORDER BY yet_another_column) AS previous_value FROM table_name;</i>
FIRST_VALUE()	Value Access	Returns the first value in the ordered result set.	<i>SELECT FIRST_VALUE(column_name) OVER (PARTITION BY another_column ORDER BY yet_another_column) AS first_value FROM table_name;</i>
LAST_VALUE()	Value Access	Returns the last value in the ordered result set.	<i>SELECT LAST_VALUE(column_name) OVER (PARTITION BY another_column ORDER BY yet_another_column ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING) AS last_value FROM table_name;</i>

Window Function - Exercises

EMP_ID	EMP_NAME	DEPT_NAME	SALARY
1	Alice	HR	50000
2	Bob	Finance	60000
3	Charlie	IT	75000
4	Diana	Finance	62000
5	Eve	HR	52000
6	Frank	IT	78000
7	Grace	Marketing	58000
8	Hank	HR	49000
9	Ivy	Finance	61000
10	Jack	IT	77000
11	Kathy	Marketing	60000
12	Leo	IT	80000
13	Mona	HR	53000
14	Nina	Finance	64000
15	Oscar	Marketing	55000

Questions:

- Write a SQL query to extract all columns from an employee table along with the maximum salary across all employees in a new column MAX_SALARY.
- Modify the query to filter employees whose salary is greater than 70,000.
- Use a window function to calculate the maximum salary in the dataset and include it as a new column in the result.
- Write a query that combines all employee details with a calculated column showing the highest salary for any employee, while excluding employees earning 60,000 or less.

EMP_ID	EMP_NAME	DEPT_NAME	SALARY	MAX_SALARY
3	Charlie	IT	75000	80000
6	Frank	IT	78000	80000
10	Jack	IT	77000	80000
12	Leo	IT	80000	80000

```
SELECT E.*,  
MAX(SALARY) OVER () AS MAX_SALARY  
FROM EMPLOYEE_TABLE E WHERE SALARY > 70000;
```

Window Function – Exercises using ROW_NUMBER () & OVER

Question

- Write a query to retrieve the top two employees (based on EMP_ID in descending order) from each department.
- How does the ROW_NUMBER() function with PARTITION BY DEPT_NAME help in ranking employees within each department?
- Modify a query to include a ranking column for employees within each department, ordered by EMP_ID in descending order.
- Explain the purpose of filtering rows using WHERE X.RN < 3 in the query.
- Write a query that selects all details of the top two employees in each department based on their EMP_ID.

```
SELECT * FROM (  
    SELECT E.*,  
        ROW_NUMBER() OVER (PARTITION BY DEPT_NAME ORDER BY EMP_ID DESC) AS RN  
    FROM EMPLOYEE_TABLE E) X  
WHERE X.RN < 3;
```

EMP_NAME	DEPT_NAME	SALARY	RN	EMP_ID
Nina	Finance	64000	1	14
Ivy	Finance	61000	2	9
Mona	HR	53000	1	13
Hank	HR	49000	2	8
Leo	IT	80000	1	12
Jack	IT	77000	2	10
Oscar	Marketing	55000	1	15
Kathy	Marketing	60000	2	11



Window Function – Exercises using ROW_NUMBER & OVER

Question

- Write a query to retrieve the top two employees (based on EMP_ID in descending order) from each department.
- How does the ROW_NUMBER() function with PARTITION BY DEPT_NAME help in ranking employees within each department?
- Modify a query to include a ranking column for employees within each department, ordered by EMP_ID in descending order.
- Explain the purpose of filtering rows using WHERE X.RN < 3 in the query.
- Write a query that selects all details of the top two employees in each department based on their EMP_ID.

```
SELECT * FROM (  
    SELECT E.*,  
        ROW_NUMBER() OVER (PARTITION BY DEPT_NAME ORDER BY EMP_ID DESC) AS RN  
    FROM EMPLOYEE_TABLE E) X  
WHERE X.RN < 3;
```

EMP_NAME	DEPT_NAME	SALARY	RN	EMP_ID
Nina	Finance	64000	1	14
Ivy	Finance	61000	2	9
Mona	HR	53000	1	13
Hank	HR	49000	2	8
Leo	IT	80000	1	12
Jack	IT	77000	2	10
Oscar	Marketing	55000	1	15
Kathy	Marketing	60000	2	11



PROBLEM SOLVING

Window Function – Exercises: FIRST_VALUE()

Question

- Write a query to display each employee's details along with the first salary in their department using the FIRST_VALUE function.
- How does the FIRST_VALUE function help retrieve the earliest salary in each department?

```
SELECT E.*,  
       FIRST_VALUE(SALARY) OVER (PARTITION BY DEPT_NAME ORDER BY EMP_ID ASC) AS  
FIRST_SALARY  
FROM EMPLOYEE_TABLE E;
```



EMP_NAME	DEPT_NAME	EMP_ID	SALARY	AVG_SALARY
Bob	Finance	2	60000	60000
Diana	Finance	4	62000	60000
Ivy	Finance	9	61000	60000
Nina	Finance	14	64000	60000
Alice	HR	1	50000	50000
Eve	HR	5	52000	50000
Hank	HR	8	49000	50000
Mona	HR	13	53000	50000
Charlie	IT	3	75000	75000
Frank	IT	6	78000	75000
Jack	IT	10	77000	75000
Leo	IT	12	80000	75000
Grace	Marketing	7	58000	58000
Kathy	Marketing	11	60000	58000
Oscar	Marketing	15	55000	58000

Window Function – Exercises: DENSE_RANK()

Question

- Write a query to rank employees by salary in their department, ensuring no gaps in rank values for ties, using DENSE_RANK().
- Explain the difference between RANK and DENSE_RANK() in ranking employees.

```
SELECT E.*,  
       DENSE_RANK() OVER (PARTITION BY DEPT_NAME ORDER BY SALARY DESC) AS  
       SALARY_RANK  
FROM EMPLOYEE_TABLE E;
```

EMP_ID	EMP_NAME	DEPT_NAME	SALARY	SALARY_RANK
14	Nina	Finance	64000	1
4	Diana	Finance	62000	2
9	Ivy	Finance	61000	3
2	Bob	Finance	60000	4
13	Mona	HR	53000	1
5	Eve	HR	52000	2
1	Alice	HR	50000	3
8	Hank	HR	49000	4
12	Leo	IT	80000	1
6	Frank	IT	78000	2
10	Jack	IT	77000	3
3	Charlie	IT	75000	4
11	Kathy	Marketing	60000	1
7	Grace	Marketing	58000	2
15	Oscar	Marketing	55000	3



Window Function – Exercises: NTILE()

Question

- Use the NTILE function to divide employees into 4 salary quartiles and display the quartile number for each employee.
- How does NTILE partition employees into groups based on salary?

```
SELECT E.*,  
       NTILE(4) OVER (ORDER BY SALARY DESC) AS SALARY_QUARTILE  
FROM EMPLOYEE_TABLE E;
```



EMP_ID	EMP_NAME	DEPT_NAME	SALARY	CATEGORY
12	Leo	IT	80000	1
6	Frank	IT	78000	1
10	Jack	IT	77000	1
3	Charlie	IT	75000	1
14	Nina	Finance	64000	2
4	Diana	Finance	62000	2
9	Ivy	Finance	61000	2
2	Bob	Finance	60000	2
11	Kathy	Marketing	60000	3
7	Grace	Marketing	58000	3
15	Oscar	Marketing	55000	3
13	Mona	HR	53000	3
5	Eve	HR	52000	4
1	Alice	HR	50000	4
8	Hank	HR	49000	4

Window Function – Exercises: LAG()

Question

For LEAD and LAG

- Use the **LEAD** function to display each employee's salary along with the next higher salary within the same department.
- Write a query using the **LAG** function to show each employee's salary along with the previous salary within their department.

```
SELECT E.*,  
       LAG(SALARY) OVER (PARTITION BY DEPT_NAME ORDER BY SALARY DESC) AS  
       PREVIOUS_SALARY  
FROM EMPLOYEE_TABLE E;
```

EMP_ID	EMP_NAME	DEPT_NAME	SALARY	PREV_SALARY
14	Nina	Finance	64000	NULL
4	Diana	Finance	62000	64000
9	Ivy	Finance	61000	62000
2	Bob	Finance	60000	61000
13	Mona	HR	53000	NULL
5	Eve	HR	52000	53000
1	Alice	HR	50000	52000
8	Hank	HR	49000	50000
12	Leo	IT	80000	NULL
6	Frank	IT	78000	80000
10	Jack	IT	77000	78000
3	Charlie	IT	75000	77000
11	Kathy	Marketing	60000	NULL
7	Grace	Marketing	58000	60000
15	Oscar	Marketing	55000	58000



Aggregate Functions – Product Table



ProductID	Name	ListPrice	Production	Quantity
1	Widget A	25.00	10-JUL-23	100
2	Gadget B	45.00	15-SEP-23	200
3	Device C	35.50	01-NOV-23	150
4	Tool D	60.00	05-DEC-23	300
5	Machine E	120.75	20-JAN-24	50
6	Gizmo F	80.99	10-MAR-24	120
7	Appliance G	90.50	15-MAY-24	90

```
SELECT
  ROUND(SUM(ListPrice * Quantity), 2) AS TotalValue,  -- Total value of all products (rounded to 2
decimals)
  ROUND(AVG(ListPrice), 2) AS AveragePrice,           -- Average list price (rounded to 2 decimals)
  COUNT(ProductID) AS ProductCount,                 -- Total number of products
  ROUND(MIN(ListPrice), 2) AS MinimumPrice,          -- Minimum list price (rounded to 2 decimals)
  ROUND(MAX(ListPrice), 2) AS MaximumPrice           -- Maximum list price (rounded to 2 decimals)
FROM Production;
```

TotalValue	AveragePrice	ProductCount	MinimumPrice	MaximumPrice
58726.3	65.39	7	25	120.75

Aggregate Functions – Production Table

```
SELECT
  EXTRACT(MONTH FROM Production) AS Month,
  AVG(ListPrice) AS AvgPrice
FROM
  Production
GROUP BY
  EXTRACT(MONTH FROM Production)
HAVING
  AVG(ListPrice) > 50;
```

Month	AvgPrice
12	60
1	120.75
3	80.99
5	90.5

```
SELECT
  EXTRACT(MONTH FROM Production) AS Month,
  SUM(ListPrice * Quantity) AS TotalPrice
FROM
  Production
GROUP BY
  EXTRACT(MONTH FROM Production)
HAVING
  SUM(ListPrice * Quantity) > 5000;
```

ID	Value
9	9000
11	5325
12	18000
1	6037.5
3	9718.8
5	8145

90th Percentile of TotalPrice

```
SELECT
  EXTRACT(MONTH FROM Production) AS Month,
  PERCENTILE_CONT(0.90) WITHIN GROUP (ORDER BY ListPrice * Quantity) AS NinetyPercentilePrice
FROM
  Production
GROUP BY
  EXTRACT(MONTH FROM Production);
```

Month	NinetyPercentilePrice
1	6037.5
3	9718.8
5	8145
7	2500
9	9000
11	5325
12	18000

Running Total of TotalPrice Over Months



```
SELECT
  EXTRACT(MONTH FROM Production) AS Month,
  SUM(ListPrice * Quantity) AS TotalPrice,
  SUM(SUM(ListPrice * Quantity)) OVER (ORDER BY EXTRACT(MONTH FROM Production))
  AS RunningTotal
FROM
  Products
GROUP BY
  EXTRACT(MONTH FROM Product)
ORDER BY
  Month;
```

Month	TotalPrice	RunningTotal
1	6037.5	6037.5
3	9718.8	15756.3
5	8145	23901.3
7	2500	26401.3
9	9000	35401.3
11	5325	40726.3
12	18000	58726.3

Multi-Level Aggregation: Total Quantity and Average per Product

```
WITH ProductTotals AS (  
  SELECT  
    ProductID,  
    Name,  
    SUM(Quantity) AS TotalQuantity  
  FROM  
    Production  
  GROUP BY  
    ProductID, Name  
)  
SELECT  
  ProductID,  
  Name,  
  TotalQuantity,  
  ROUND(AVG(TotalQuantity) OVER (), 2) AS AvgQuantityAcrossAllProducts  
FROM  
  ProductTotals;
```

ProductID	Name	TotalQuantity	AvgQuantityAcrossAllProducts
1	Widget A	100	144.29
2	Gadget B	200	144.29
3	Device C	150	144.29
4	Tool D	300	144.29
5	Machine E	50	144.29
6	Gizmo F	120	144.29
7	Appliance G	90	144.29



```
SELECT  
  EXTRACT(MONTH FROM Production) AS Month,  
  CASE  
    WHEN ListPrice < 50 THEN 'Low'  
    WHEN ListPrice BETWEEN 50 AND 100 THEN 'Medium'  
    ELSE 'High'  
  END AS PriceRange,  
  SUM(ListPrice * Quantity) AS TotalPrice  
FROM  
  Production  
GROUP BY  
  EXTRACT(MONTH FROM Production),  
  CASE  
    WHEN ListPrice < 50 THEN 'Low'  
    WHEN ListPrice BETWEEN 50 AND 100 THEN 'Medium'  
    ELSE 'High'  
  END;
```

Grouping by Price Range and Month

Month	PriceRange	TotalPrice
7	Low	2500
9	Low	9000
11	Low	5325
12	Medium	18000
1	High	6037.5
3	Medium	9718.8

Question

Write a SQL query to extract the month from the SaleDate, categorize the Price into 'Low', 'Medium', and 'High' ranges, calculate the total sales revenue (Price * QuantitySold) for each month and price range, and display the results with Month, PriceRange, and TotalSales.

SQL Subqueries



A subquery in SQL is a query nested inside another SQL query, often used to perform operations that need a result from a secondary query to complete the primary one. Here's a refined explanation based on your input:

Definition of a Subquery

- A **subquery** is a SQL query embedded within another SQL statement, often referred to as an "inner query" or "inner select," while the main query containing it is called the "outer query" or "outer select."
- The inner query executes first, and its result is then used by the outer query.

Locations of a Subquery

Subqueries can be placed in several parts of a SQL statement:

- **SELECT clause:** For calculating values to be used in the result set.
- **FROM clause:** As a derived table.
- **WHERE clause:** To filter rows based on criteria from the inner query.
- **HAVING clause:** To filter groups.

```
SELECT ProductID,  
       Name,  
       ListPrice  
FROM   production.Product  
WHERE  ListPrice > (SELECT AVG(ListPrice)  
                   FROM   Production.Product)
```

subquery

```
SELECT  select_list  
FROM    table  
WHERE   expr operator  
  
        (SELECT  select_list  
         FROM    table);
```

- The subquery (inner query) executes once before the main query (outer query) executes.
- The main query (outer query) use the subquery result.

SQL Subqueries – Use cases

Use Cases of Subqueries

Subqueries can be used inside SELECT, INSERT, UPDATE, or DELETE statements to:

- **Compare an expression:** e.g., WHERE column_name = (SELECT ...).
- **Determine inclusion:** e.g., WHERE column_name IN (SELECT ...).
- **Check existence:** e.g., WHERE EXISTS (SELECT ...).

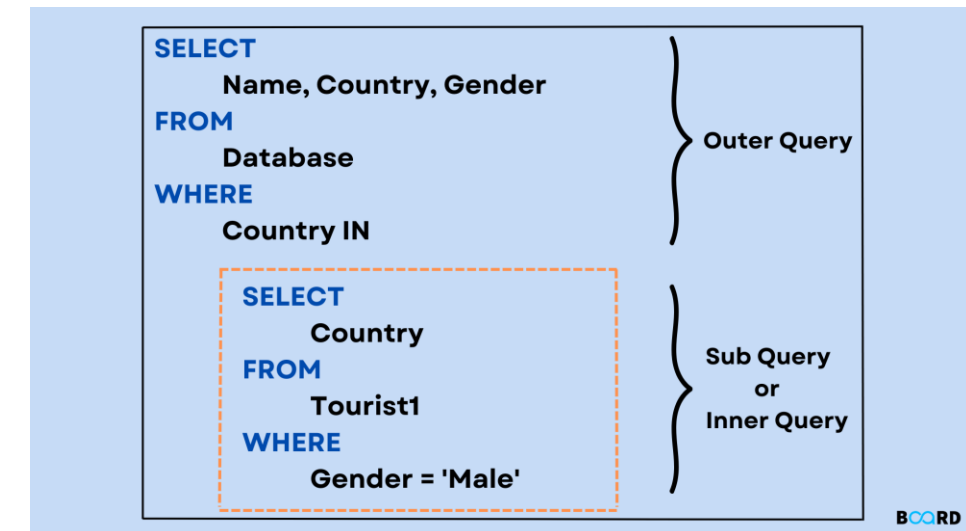
Comparison Operators in Subqueries

- Subqueries can use comparison operators like =, >, <, as well as multiple-row operators such as IN, ANY, or ALL.

Practical Functions of Subqueries

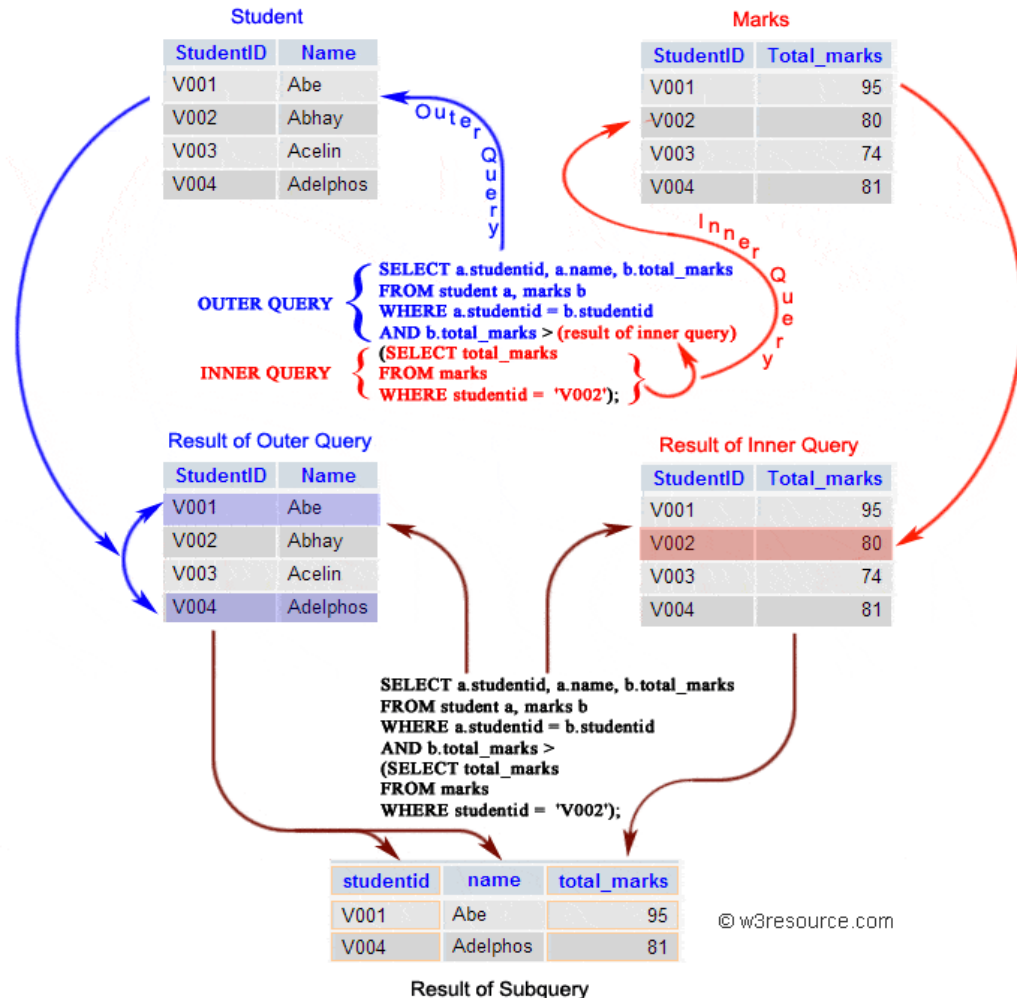
1. **Compare an expression** to the result of another query.
2. **Determine if an expression is included** in the result set.
3. **Check whether the subquery returns any rows**, which can influence the outer query's behavior.

Subqueries are powerful tools for breaking down complex queries into manageable pieces, enabling efficient data filtering, comparison, and transformation.



Source Image: <https://www.boardinfinity.com/blog/subquery-in-sql/>

Subqueries: Guidelines



There are some guidelines to consider when using subqueries :

- A subquery must be enclosed in parentheses.
- A subquery must be placed on the right side of the comparison operator.

Subqueries cannot manipulate their results internally, therefore ORDER BY clause cannot be added into a subquery. You can use an ORDER BY clause in the main SELECT statement (outer query) which will be the last clause.

- Use single-row operators with single-row subqueries.
- If a subquery (inner query) returns a null value to the outer query, the outer query will not return any rows when using certain comparison operators in a WHERE clause.

Source Image: <https://www.w3resource.com/sql/subqueries/understanding-sql-subqueries.php>

SQL Subqueries - Exercise



Average of TotalPrice for Products with Highest Total Price in Each Month

```
SELECT
  EXTRACT(MONTH FROM Production) AS Month,
  AVG(TotalPrice) AS AvgTotalPrice
FROM (
  SELECT
    EXTRACT(MONTH FROM Production) AS Month,
    ListPrice * Quantity AS TotalPrice
  FROM
    Products
) AS SubQuery
GROUP BY
  Month
HAVING
  TotalPrice = (SELECT MAX(ListPrice * Quantity) FROM Products WHERE
  EXTRACT(MONTH FROM Production) = SubQuery.Month);
```

Month	AvgTotalPrice
1	6037.50
3	9718.80
5	8145.00

This subquery calculates the **maximum total price** (ListPrice * Quantity) for the same month as the current SubQuery.Month.

Outer Query Columns:

- EXTRACT(MONTH FROM Production) AS Month: Retrieves the **month** from the Production column.
- AVG(TotalPrice) AS AvgTotalPrice: Calculates the average of TotalPrice for each group (grouped by month).

Grouping:

- GROUP BY Month: Groups the results by the extracted Month.

Filtering with HAVING:

- The HAVING clause ensures only rows where TotalPrice equals the **maximum total price** for the same month are included. This uses a **correlated subquery**.

Using Aggregate Functions with GROUP BY and HAVING in SQL

When using aggregate functions (SUM(), AVG(), COUNT(), MIN(), MAX()), it's important to understand the role of GROUP BY and HAVING:



- **GROUP BY** is used when you want to aggregate data across different categories or groups. It combines rows with the same values in specified columns into a single row. You should apply **GROUP BY** when you need to **calculate aggregate** values (like sums or averages) for each distinct group in your data.
 - For example, if you want to calculate the total salary for each department, you would group the data by DepartmentID.
- **HAVING** is used to filter groups after aggregation has occurred. It's similar to WHERE, but while WHERE filters rows before aggregation, HAVING filters groups after they've been created by the GROUP BY clause.
 - For instance, if you want to display only departments with a total salary exceeding 500,000, you would use HAVING to apply this condition after the aggregation is done.

Key Points:

- Use GROUP BY to specify how data should be grouped before applying aggregate functions.
- Use HAVING to filter the results after the aggregation process.

Example Aggregate Functions with GROUP BY and HAVING in SQL



Example:

In the example below, the GROUP BY clause groups employees by DepartmentID, and the HAVING clause filters out any departments where the total salary is less than 500,000.

```
SELECT DepartmentID,  
        SUM(Salary) AS TotalSalary  
FROM Employees  
   GROUP BY DepartmentID -- Group employees by their department  
HAVING SUM(Salary) > 500000; -- Filter departments where total salary exceeds 500,000
```

How it's followed:

1. The **GROUP BY** clause first groups all rows (in this case, employees) by **DepartmentID**. This forms subsets (groups) of the data.
2. Aggregate functions (in this case, **SUM(Salary)**) are applied to each group, calculating the total salary for each department.
3. The **HAVING** clause then filters the results to display only those departments where the total salary exceeds 500,000.

This ensures that you can both group and filter data effectively in a single query.

Comparison: Views vs. Indexes

Feature	Views	Indexes
Definition	Virtual tables based on SELECT queries.	Data structures that optimize query performance.
Purpose	Simplify queries, enhance security, and customize data views.	Speed up data retrieval by creating quick access paths.
Data Storage	Do not store data physically.	Require additional storage for index structures.
Performance Impact	Simplifies repeated queries; does not improve performance directly.	Improves query performance, especially for large datasets.
Trade-offs	No additional storage, but depends on underlying tables for performance.	Slows down write operations and consumes more storage.
Use Cases	Simplifying reports, restricting sensitive data.	Filtering, sorting, and joining tables in large datasets.

Summary of Commands

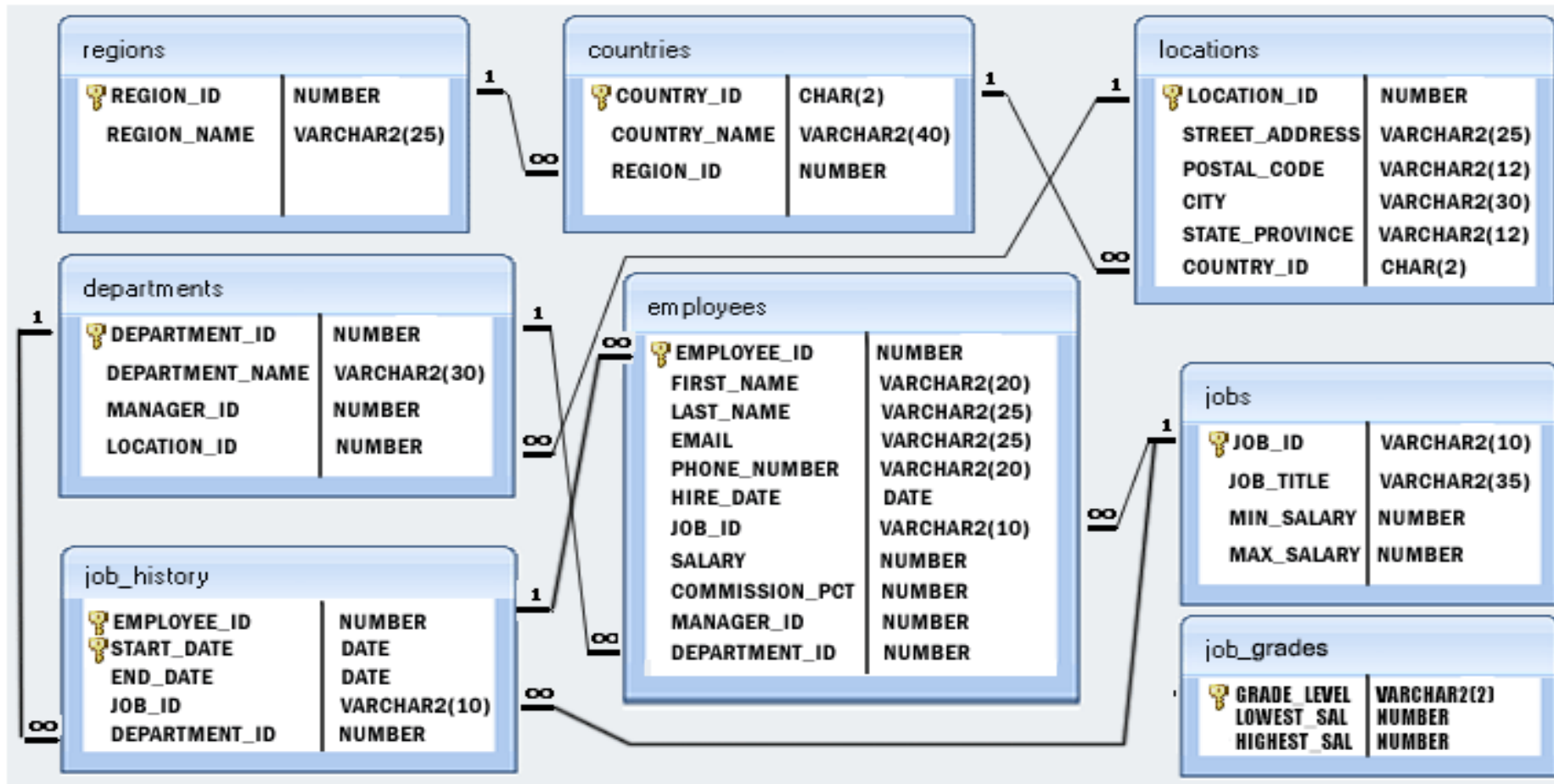
Operation	Command	Purpose
Create View	CREATE VIEW ... AS SELECT ...	Simplify complex queries or restrict data.
Alter View	CREATE OR REPLACE VIEW ...	Modify an existing view.
Drop View	DROP VIEW view_name;	Remove a view.
Create Index	CREATE INDEX ... ON table(column);	Speed up data retrieval.
Alter Index	DROP INDEX ... followed by CREATE INDEX ...	Update an existing index.
Drop Index	DROP INDEX index_name;	Delete an index.

Table that visualizes all the queries for both Indexes and Views

Operation	View	Index
Create	CREATE VIEW EmployeeView AS SELECT employee_id, firstname, department, salary FROM Employees;	CREATE INDEX idx_department ON Employees(department);
		CREATE INDEX idx_firstname_department ON Employees(firstname, department);
		CREATE INDEX idx_manager_id ON Employees(manager_id);
Update/Modify	CREATE OR REPLACE VIEW EmployeeView AS SELECT employee_id, firstname, department, salary, manager_id FROM Employees;	<i>(Indexes cannot be directly updated; they must be recreated.)</i>
Drop	DROP VIEW EmployeeView;	DROP INDEX idx_department;

Assignment I (1/4)

Use the HR database conceptual design diagram provided to answer the following questions by writing SQL queries



Assignment I (1/2)

SQL Query Development for HR Database: Table Creation, Data Manipulation, and Retrieval



Assignment: HR Database SQL Queries

Use the HR database conceptual design diagram provided to answer the following questions by writing SQL queries.

Part 1: Table Creation and Relationships

1. Create Tables

Write SQL queries to create the following tables based on the diagram:

1. *regions, countries, locations, departments, employees, jobs, job_history, job_grades*

2. Establish Relationships

Write SQL queries to create foreign key relationships between the tables:

1. *countries and regions*
2. *locations and countries*
3. *departments and locations*
4. *employees and departments*
5. *employees and jobs*
6. *job_history and employees*
7. *job_grades and jobs*

Part 2: Data Insertion

3. Insert Data

Write SQL queries to insert sample data into all tables, ensuring the foreign key constraints are respected.

Part 3: Data Retrieval

4. Basic Retrieval

Write a query to retrieve employee names, emails, job titles, and department names by joining relevant tables.

5. Employees in Specific Departments

Write a query to list employees in the 'Sales' department with their job titles and hire dates.

6. Salary Information

Write a query to find employees earning more than \$5000, displaying their names, job titles, and salaries.

7. Average Salary by Department

Write a query to calculate the average salary per department.



Assignment I (2/2)

Comprehensive SQL Querying with HR Database: Table Creation, Data Manipulation, and Advanced Retrieval



Part 4: Advanced Queries

8. Employees with Job History

List employees with past job records, including their job title, department, and previous employment duration.

9. Top Salary by Department

Write a query to find the highest-paid employee in each department.

10. Jobs and Grades

Display all jobs along with their grade levels, including the minimum and maximum salary for each job.

11. Subquery and Join

Write a query to find employees whose salary is higher than the average salary of their department. Use a subquery to calculate the department's average salary and join it with the employee's salary.



Part 5: Updates and Deletions

12. Update Salary

Increase an employee's salary by 10% for a given employee ID.

13. Delete Employees

Write a query to delete terminated employees from the database.

Assignment II: Normalization Question for Students

Problem: A university database stores student and course information in a single table:



StudentID	StudentName	CourseID	CourseName
101	Alice	C101	Database
101	Alice	C102	Networking
102	Bob	C101	Database
StudentID	StudentName	CourseID	CourseName

Question:

- Identify the issues with this table structure based on normalization principles. Which normal forms (1NF, 2NF, or 3NF) are violated? Explain why.

2. Follow-Up Task: Design normalized tables that address these violations and ensure the database adheres to 3NF. Write the SQL queries to create these tables.



Thank you!